



# Chapter 2

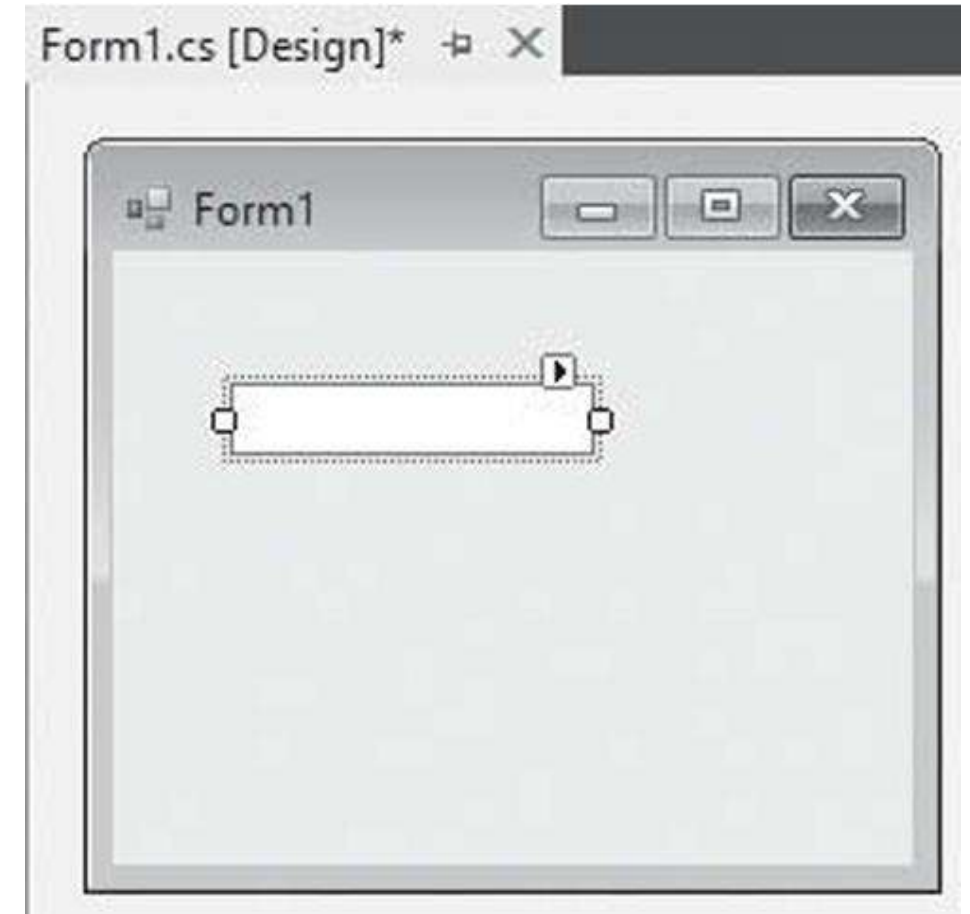
## Processing Data

By

**Dr. Sara Tarik**

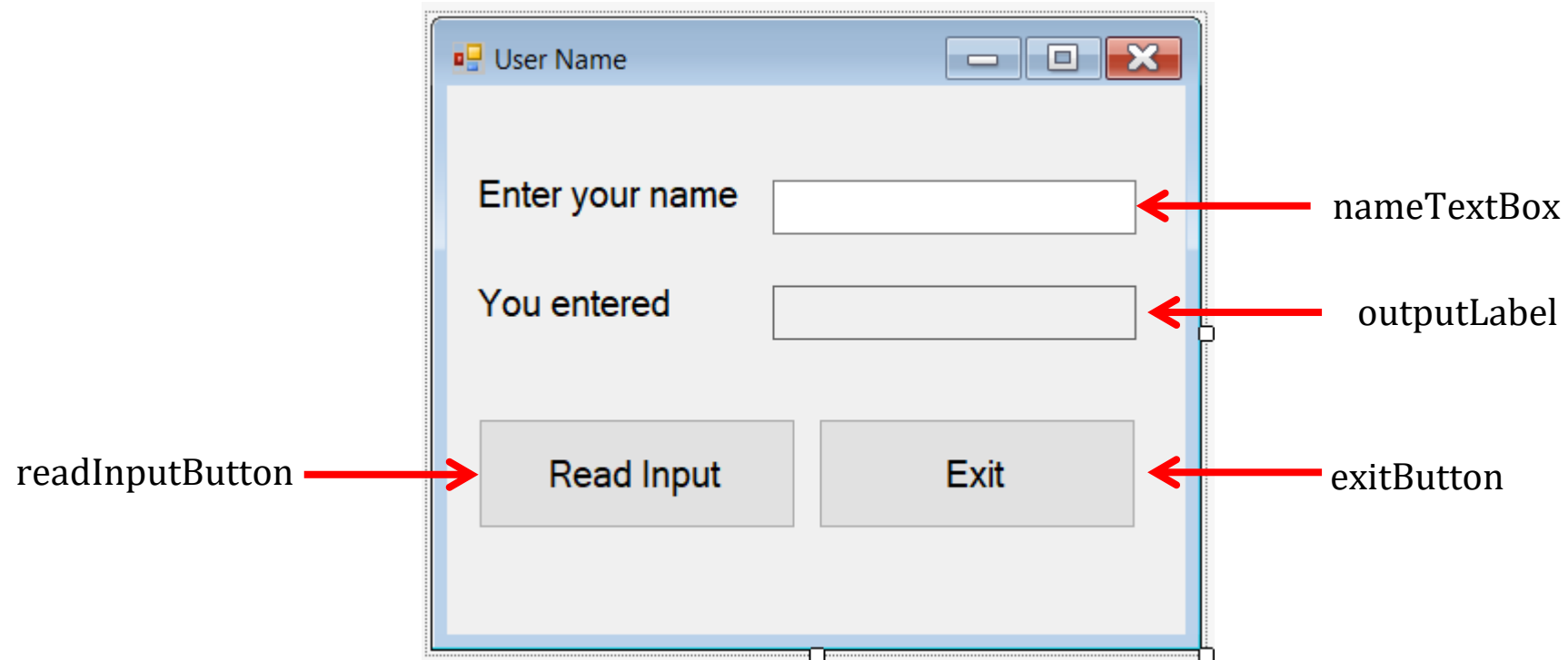
# TextBox Control

- The TextBox control is a rectangular area that can accept keyboard input from the user.
- In the Toolbox, the TextBox tool is located in the Common Controls group. When you double-click the tool, a TextBox control is created on the form.
- When the user types into a TextBox control, the text is stored in the control's Text property.
- In code, if you want to retrieve the data that has been typed into a TextBox, you simply retrieve the contents of the control's Text property. When you retrieve the contents of the Text property, you always get a string.



# Example 1

- Create a GUI application as shown below. When the user enters his name in the nameTextBox and clicks the readInputButton, the user's name is displayed in the outputLabel. When the user clicks the exitButton, the application is closed.

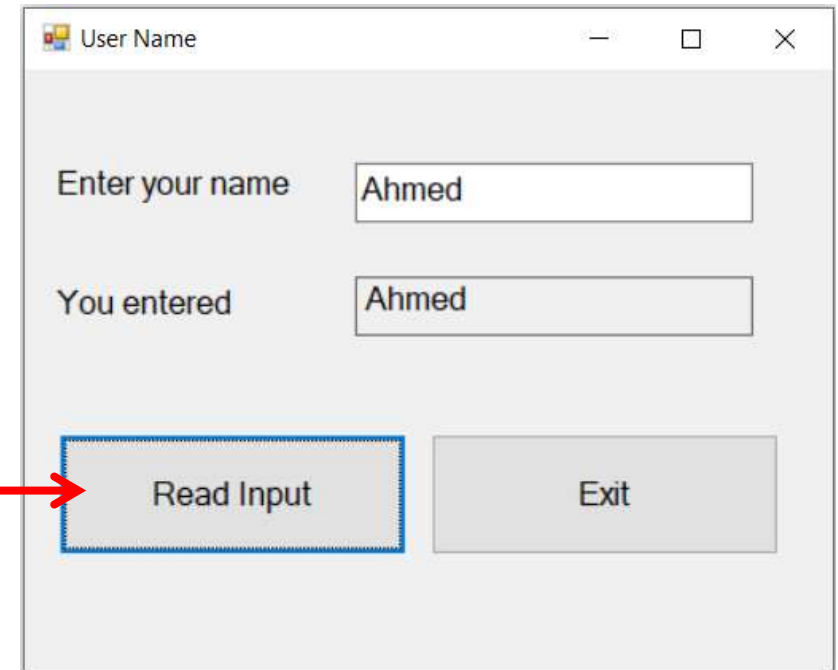


# Example 1

- The readInputButton control's Click event handler assigns the value of the nameTextBox control's Text property to the outputLabel control's Text property using the following assignment statement:

```
outputLabel.Text = nameTextBox.Text;
```

- In other words, it gets any text that has been entered by the user into the nameTextBox control and displays it in the outputLabel control.
- If you run the application, the figure shows an example of how the form appears after you have entered Ahmed and clicked the readInputButton control.



The screenshot shows a Windows application window titled "User Name". Inside the window, there are two text boxes. The first text box is labeled "Enter your name" and contains the text "Ahmed". The second text box is labeled "You entered" and also contains the text "Ahmed". Below these text boxes are two buttons: "Read Input" and "Exit". A red arrow points to the "Read Input" button, indicating that it has been clicked.

# Clearing the Contents of a TextBox Control

- You can clear the contents of a TextBox control in the same way that you clear the contents of a Label control: you assign an empty string ("" ) to the control's Text property.
- For example, the following statement clears the contents of the nameTextBox control:

```
nameTextBox.Text = "";
```

# String Concatenation

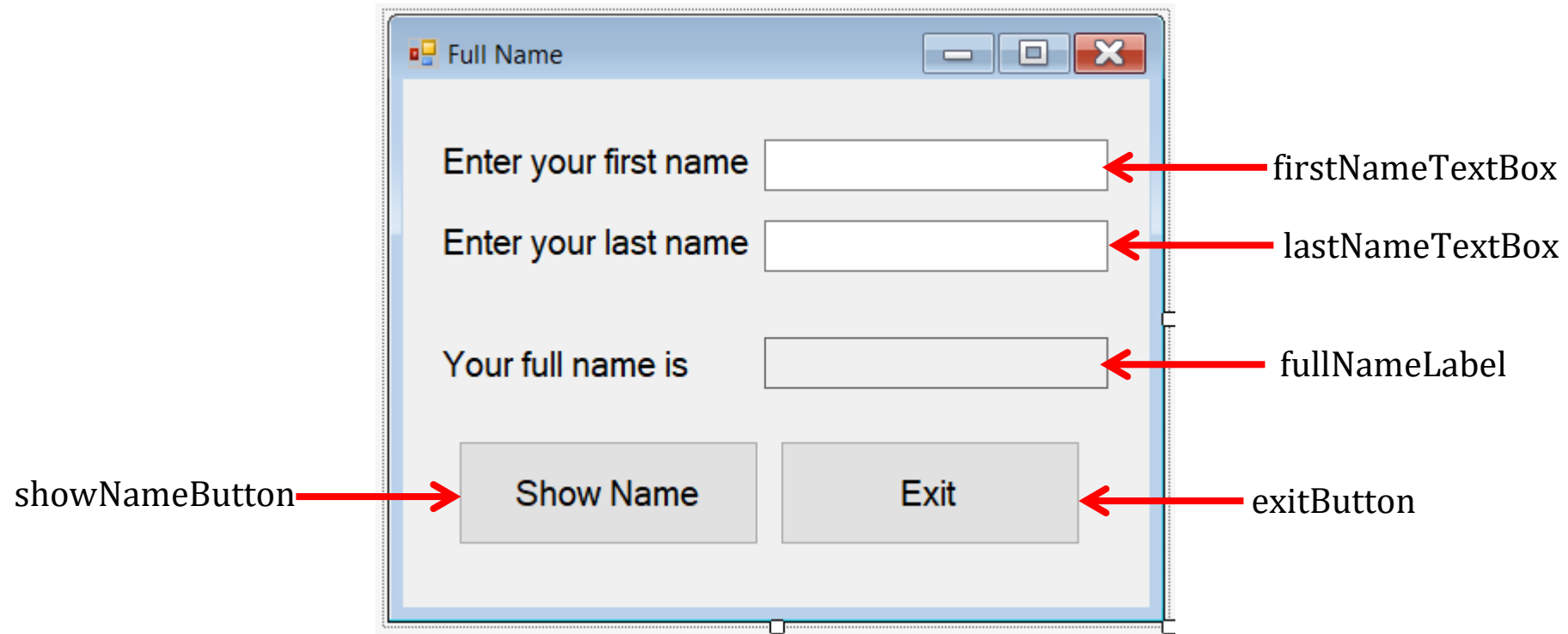
- A common operation that performed on strings is concatenation, or appending one string to the end of another string.
- In C#, the + operator is used to concatenate strings.
- **Example:**

```
string message;  
  
message = "Hello " + "world";  
  
MessageBox.Show(message);
```

- When the message box is displayed, it shows the string **Hello world**.

# Example 2

- Create a GUI application as shown below. When the user enters his first name into the firstNameTextBox, last name into the lastNameTextBox, and clicks the showNameButton, the user's full name is displayed in the fullNameLabel. When the user clicks the exitButton, the application is closed.



# Example 2

■ Write the statements shown on the right in the showNameButton\_Click event handler:

- Statement 1 is a variable declaration. It declares a string variable named fullName.
- Statement 2 assigns the result of a string concatenation to the fullName variable. The string that is assigned to the variable begins with the value of the firstNameTextBox control's Text property, followed by a space (" "), followed by the value of the lastNameTextBox control's Text property.

```
public partial class Form7 : Form
{
    0 references
    public Form7()
    {
        InitializeComponent();
    }

    1 reference
    private void showNameButton_Click(object sender, EventArgs e)
    {
        1 → string fullName;
        2 → fullName = firstNameTextBox.Text + " " + lastNameTextBox.Text;
        3 → fullNameLabel.Text = fullName;
    }

    1 reference
    private void exitButton_Click(object sender, EventArgs e)
    {
        this.Close();
    }
}
```

# Example 2

For example, if the user has entered Ahmed into the firstNameTextBox control and Atif into the lastNameTextBox control, this statement will assign the string "Ahmed Atif" to the fullName variable.

- Statement 3 assigns the fullName variable to the fullNameLabel control's Text property. As a result, the string that is stored in the fullName variable is displayed in the fullNameLabel control.

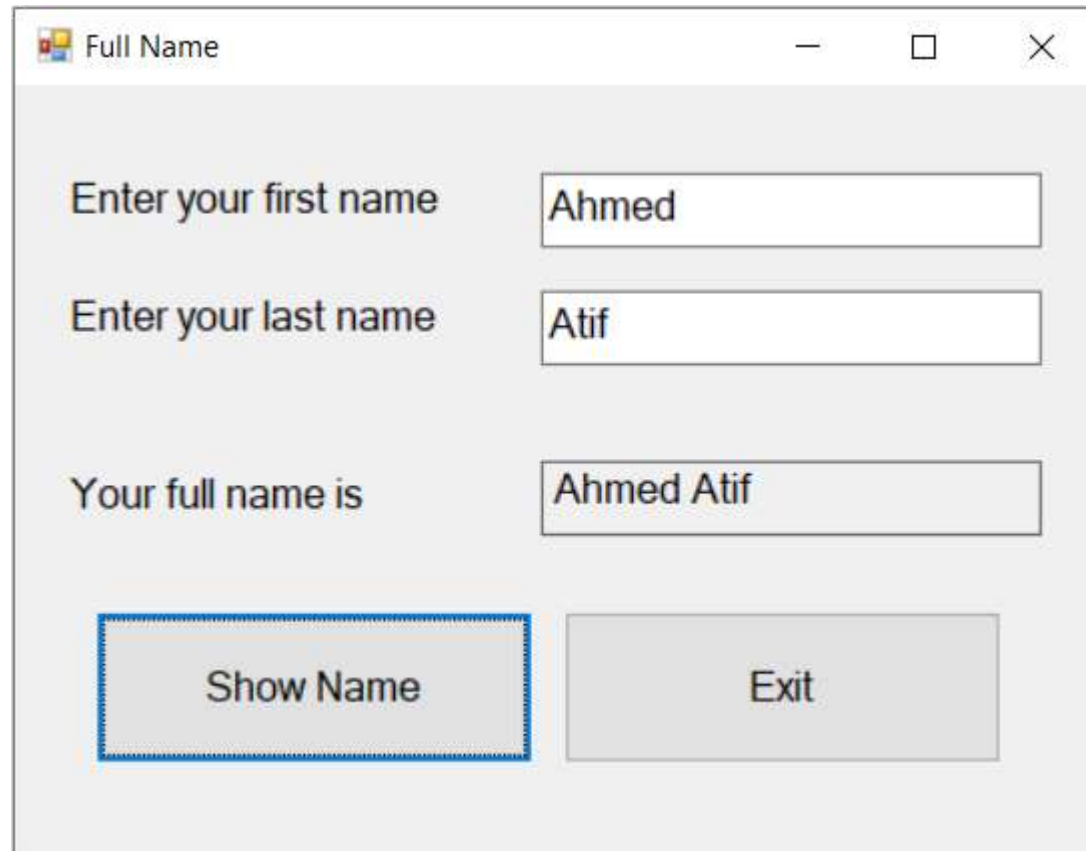
```
public partial class Form7 : Form
{
    0 references
    public Form7()
    {
        InitializeComponent();
    }

    1 reference
    private void showNameButton_Click(object sender, EventArgs e)
    {
        1 → string fullName;
        2 → fullName = firstNameTextBox.Text + " " + lastNameTextBox.Text;
        3 → fullNameLabel.Text = fullName;
    }

    1 reference
    private void exitButton_Click(object sender, EventArgs e)
    {
        this.Close();
    }
}
```

# Example 2

- This figure shows an example of how the form appears after you run the application and entered Ahmed for the first name and Atif for the last name and clicked the “Show Name” button control.



Full Name

Enter your first name Ahmed

Enter your last name Atif

Your full name is Ahmed Atif

Show Name Exit

# Local Variables

- The fullName variable is declared inside the event handler method.
- Variables that are declared inside a method are known as local variables.
- A local variable belongs to the method in which it is declared, and only statements inside that method can access the variable.
- An error will occur if a statement in one method tries to access a local variable that belongs to another method, as shown in the code.

```
public partial class Form7 : Form
{
    1 reference
    public Form7()
    {
        InitializeComponent();
    }

    1 reference
    private void showNameButton_Click(object sender, EventArgs e)
    {
        string fullName;
        fullName = firstNameTextBox.Text + " " + lastNameTextBox.Text;
    }

    1 reference
    private void exitButton_Click(object sender, EventArgs e)
    {
        fullNameLabel.Text = fullName; ← ERROR!
        this.Close();
    }
}
```

# Scope of a Variable

- Programmers use the term scope to describe the part of a program in which a variable may be accessed.
- A variable is visible only to statements inside the variable's scope.
- A local variable's scope begins at the variable's declaration and ends at the end of the method in which the variable is declared.

# Duplicate Variable Names

- Note that you cannot declare multiple variables with the same name within the same method or event handler. For example, if you declare a variable named `productDescription` in an event handler, you cannot declare another variable with that name in the same event handler.
- However, you can have variables of the same name declared in different methods.
- When you assign a value to a variable, that value will remain in the variable until you assign a different value to this variable.
- The output of the code shown on the right is a message box displaying “Chocolate Truffle”.

```
1 // Declare a string variable.
2 string productDescription;
3
4 // Assign a value to the variable.
5 productDescription = "Large Medium-Roast Coffee";
6
7 // Display the variable's value.
8 MessageBox.Show(productDescription);
9
10 // Assign a different value to the variable.
11 productDescription = "Chocolate Truffle";
12
13 // Display the variable's value.
14 MessageBox.Show(productDescription);
```

# Assignment Compatibility

- You can assign a value to a variable only if the value is compatible with the variable's data type.
- For example, the following code will not work because it attempts to assign a nonstring value to a string variable:

```
string employeeID;
```

```
employeeID = 125;
```

**ERROR!**

- Instead, you can store the string literal "125" in a string variable.

# Example 3

- Create a GUI application that lets the user enter information about his or her birthdate such as the day of the week (Monday, Tuesday, etc.), the name of the month (January, February, etc.), the numeric day of the month, and the year. When the user clicks the “Show Date” button, the application concatenates the contents of the TextBoxes into a string such as Friday, June 1, 1990 and displays this string in the dateOutputLabel control. When the user clicks the Clear button, the contents of the TextBoxes and the dateOutputLabel control are cleared. The “Exit” button closes the application’s form.

The image displays two side-by-side screenshots of a Windows-style GUI application titled "Birth Date String".

The left screenshot shows the initial state of the application. It features four input fields with the following labels: "Enter the day of the week", "Enter the name of the month", "Enter the numeric day of the month", and "Enter the year". Below these fields is a large empty text box. At the bottom are three buttons: "Show Date", "Clear", and "Exit".

The right screenshot shows the application after data has been entered. The input fields now contain "Friday", "June", "1", and "1990". The large text box now displays the concatenated string "Friday, June 1, 1990". A mouse cursor is shown clicking the "Show Date" button, which is highlighted with a dashed border.

# Example 3

- 1) Set up the application's form as shown below.

Form1.cs [Design] X

Birth Date String

dayOfWeekPromptLabel → Enter the day of the week → dayOfWeekTextBox

monthPromptLabel → Enter the name of the month → monthTextBox

dayOfMonthPromptLabel → Enter the numeric day of the month → dayOfMonthTextBox

yearPromptLabel → Enter the year → yearTextBox

dateOutputLabel →

showDateButton → Show Date

clearButton → Clear

exitButton → Exit

# Example 3

2) Set up the properties of the controls as shown in the following table.

| Control Name          | Control Type | Property Settings                                                                                                                                                 |
|-----------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| dayOfWeekPromptLabel  | Label        | <b>Text:</b> <i>Enter the day of the week</i>                                                                                                                     |
| monthPromptLabel      | Label        | <b>Text:</b> <i>Enter the name of the month</i>                                                                                                                   |
| dayOfMonthPromptLabel | Label        | <b>Text:</b> <i>Enter the numeric day of the month</i>                                                                                                            |
| yearPromptLabel       | Label        | <b>Text:</b> <i>Enter the year</i>                                                                                                                                |
| dayOfWeekTextBox      | TextBox      | No properties changed                                                                                                                                             |
| monthTextBox          | TextBox      | No properties changed                                                                                                                                             |
| dayOfMonthTextBox     | TextBox      | No properties changed                                                                                                                                             |
| yearTextBox           | TextBox      | No properties changed                                                                                                                                             |
| dateOutputLabel       | Label        | <b>AutoSize:</b> False<br><b>BorderStyle:</b> FixedSingle<br><b>Text:</b> (The contents of the Text property have been erased.)<br><b>TextAlign:</b> MiddleCenter |
| showDateButton        | Button       | <b>Text:</b> <i>Show Date</i>                                                                                                                                     |
| clearButton           | Button       | <b>Text:</b> <i>Clear</i>                                                                                                                                         |
| exitButton            | Button       | <b>Text:</b> <i>Exit</i>                                                                                                                                          |

# Example 3

- 3) Write the statement that you want to execute inside the event handler of Button controls, as shown on the right.

```
private void showDateButton_Click(object sender, EventArgs e)
{
    // Declare a string variable.
    string output;

    // Concatenate the input and build the output string.
    output = dayOfWeekTextBox.Text + ", " +
        monthTextBox.Text + " " +
        dayOfMonthTextBox.Text + ", " +
        yearTextBox.Text;

    // Display the output string in the Label control.
    dateOutputLabel.Text = output;
}

private void clearButton_Click(object sender, EventArgs e)
{
    // Clear the TextBoxes.
    dayOfWeekTextBox.Text = "";
    monthTextBox.Text = "";
    dayOfMonthTextBox.Text = "";
    yearTextBox.Text = "";

    // Clear the dateOutputLabel control.
    dateOutputLabel.Text = "";
}

private void exitButton_Click(object sender, EventArgs e)
{
    // Close the form.
    this.Close();
}
```

# Example 3

- 4) Run the application and click the buttons to view the output. The form will appear as shown on the left side of the figure. Test the application by entering values into the TextBoxes and clicking the “Show Date” button. The date should be displayed as shown on the right side of the figure. Click the “Clear” button to erase the contents of the TextBoxes and the Label control. Finally, click the “Exit” button to close the form.

The figure displays two side-by-side screenshots of a Windows application window titled "Birth Date String".

The left screenshot shows the initial state of the form with four empty text boxes for input:

- Enter the day of the week
- Enter the name of the month
- Enter the numeric day of the month
- Enter the year

Below the input fields is a large empty text box for the output. At the bottom are three buttons: "Show Date", "Clear", and "Exit".

The right screenshot shows the form after data has been entered and the "Show Date" button has been clicked. The input fields now contain:

- Friday
- June
- 1
- 1990

The output text box now displays "Friday, June 1, 1990". A mouse cursor is pointing at the "Show Date" button, which is highlighted with a dashed border.

# Declaring Multiple Variables with One Statement

- Declare multiple variables of the same data type with one declaration statement:

```
string lastName, firstName, middleName;
```

- Declare and initialize the variables with one statement:

```
string lastName = "Jones", firstName = "Jill", middleName = "Rebecca";
```

**Or**

```
string lastName = "Jones",
```

```
firstName = "Jill",
```

```
middleName = "Rebecca";
```

# int Variables Assignment Compatibility

- You can assign int values to int variables, but you cannot assign double or decimal values to int variables.
- For example:

```
int hoursWorked = 40;      This works
```

```
int unitsSold = 650m;      ERROR!
```

```
int score = -25.5;      ERROR!
```

# double Variables Assignment Compatibility

- You can assign either double or int values to double variables, but you cannot assign decimal values to double variables. For example:

```
double distance = 28.75; This works
```

```
double speed = 75; This works
```

```
double sales = 6500.0m; ERROR!
```

- You are allowed to assign an int value to a double variable because any number that can be stored as an int can be converted to a double with no loss of data. When you assign an int value to a double variable, the int value is implicitly converted to a double.
- However, you cannot assign a decimal value to a double variable because the decimal data type allows for much greater precision than the double data type.

# decimal Variables Assignment Compatibility

- You can assign either decimal or int values to decimal variables, but you cannot assign double values to decimal variables.
- For example:

```
decimal balance = 9280.73m; This works
```

```
decimal price = 50; This works
```

```
decimal sales = 6500.0; ERROR!
```

# Converting Values with Cast Operators

- You can use a cast operator to explicitly convert a value from one numeric data type to another, even if the conversion might result in a loss of data.
- A cast operator is the name of the desired data type, written inside parentheses and placed to the left of the value that you want to convert.
- In the following example, a cast operator is used to convert the value of `realNumber` to an `int`. Then, the converted value is assigned to `wholeNumber` variable. After this statement executes, the `wholeNumber` variable is assigned the value 3.

```
int wholeNumber;  
  
double realNumber = 3.0;  
  
wholeNumber = (int)realNumber;
```

# Converting Values with Cast Operators

- The following table shows examples of uses cast operator.

| Code Example                                                                                           | Description                                                                                                                                          |
|--------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre>int wholeNumber;<br/>decimal moneyNumber = 4500m;<br/>wholeNumber = (int)moneyNumber;</pre>       | The (int) cast operator converts the value of the moneyNumber variable to an int. The converted value is assigned to the wholeNumber variable.       |
| <pre>double realNumber;<br/>decimal moneyNumber = 625.70m;<br/>realNumber = (double)moneyNumber;</pre> | The (double) cast operator converts the value of the moneyNumber variable to a double. The converted value is assigned to the realNumber variable.   |
| <pre>decimal moneyNumber;<br/>double realNumber = 98.9;<br/>moneyNumber = (decimal)realNumber;</pre>   | The (decimal) cast operator converts the value of the realNumber variable to a decimal. The converted value is assigned to the moneyNumber variable. |

# Mixing Data Types in a Math Expression

- When you perform a math operation on two operands, the data type of the result will depend on the data type of the operands.
- If the operands are of the same data type, the result will also be of that data type.
- For example:
  - When an operation is performed on two int values, the result will be an int.
  - When an operation is performed on two double values, the result will be a double.
  - When an operation is performed on two decimal values, the result will be a decimal.
- It's not uncommon, however, for a math expression to have operands of different data types.

# Mixing Data Types in a Math Expression

- C# handles operations involving int, double, and decimal operands in the following ways:
  - When a math expression involves an **int** and a **double**, the int is temporarily converted to a double, and the result is a **double**.
  - When a math expression involves an **int** and a **decimal**, the int is temporarily converted to a decimal, and the result is a **decimal**.
  - Math expressions involving a **double** and a **decimal** are **not allowed** unless a cast operator is used to convert one of the operands.

# Mixing Data Types in a Math Expression

## ■ Example 1:

```
int hoursWorked;  
  
decimal payRate;  
  
decimal grossPay;  
  
grossPay = hoursWorked * payRate;
```

- The math expression on the right side of the = operator multiplies an **int** by a **decimal**.
- When the statement executes, the value of the **hoursWorked** variable is temporarily converted to a decimal and then multiplied by the **payRate** variable.
- The result is a **decimal** and is assigned to the **grossPay** variable.

# Mixing Data Types in a Math Expression

## ■ Example 2:

```
double weight;  
  
decimal pricePerPound;  
  
decimal total;  
  
total = weight * pricePerPound; ERROR!
```

- The compiler will not allow this statement because **weight** is a double and **pricePerPound** is a decimal. To fix the statement, you can insert a cast operator, as shown here:

```
total = (decimal)weight * pricePerPound;
```

# Inputting and Outputting Numeric Values

- If the user has entered a number into a TextBox, the number will be stored as a string in the TextBox's Text property.
- If you want to store that number in a numeric variable, you have to convert it to the appropriate numeric data type.
- When you want to display the value of a numeric variable in a Label control or a message box, you have to convert it to a string.

# Getting a Number from a TextBox

- Graphical User Interface (GUI) applications typically use TextBox controls to read keyboard input.
- Any data that the user enters into a TextBox control is stored in the control's Text property as a string, even if it is a number.
- For example, if the user enters the number 72 into a TextBox control, the input is stored as the string "72" in the control's Text property
- If the user has entered a numeric value into a TextBox control and you want to assign that value to a numeric variable, you have to convert the control's Text property to the desired numeric data type.
- Unfortunately, you cannot use a cast operator to convert a string to a numeric type.

# Getting a Number from a TextBox

- To convert a string to any of the numeric data types, we use a family of methods in the .NET Framework known as the Parse methods.
- The Parse methods are used to convert a string to a specific data type:
  - **int.Parse** method: convert a string to an int.
  - **double.Parse** method: convert a string to a double.
  - **decimal.Parse** method: convert a string to a decimal.

# Getting a Number from a TextBox

- **Example 1:** Assume that hoursWorkedTextBox is the name of a TextBox control. Use the int.Parse method to convert a control's Text property to an int, as shown below.

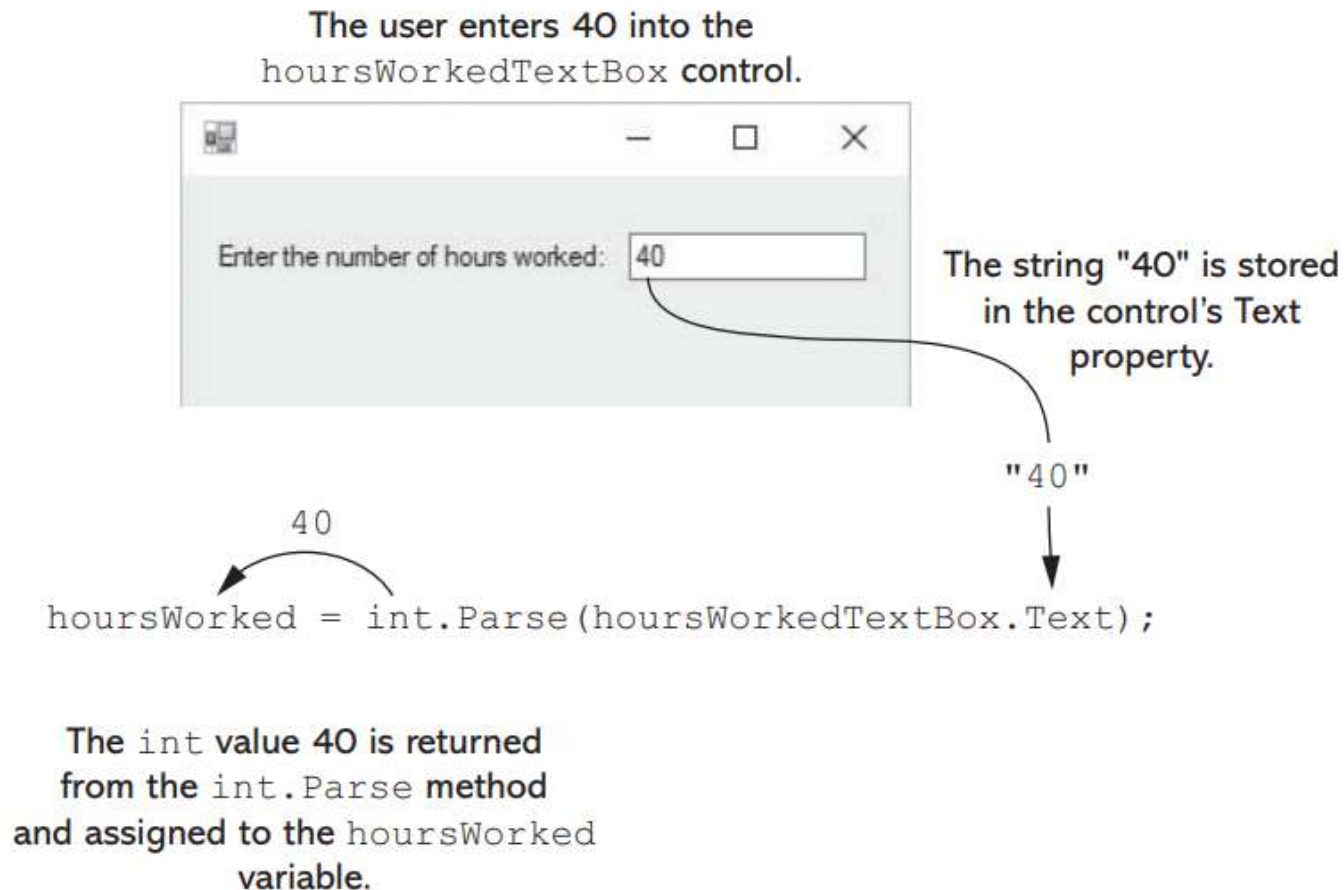
```
int hoursWorked;  
  
hoursWorked = int.Parse(hoursWorkedTextBox.Text);
```

- **Example 2:** Assume that temperatureTextBox is the name of a TextBox control. The following code sample demonstrates the double.Parse method.

```
double temperature;  
  
temperature = double.Parse(temperatureTextBox.Text);
```

# Getting a Number from a TextBox

- Let's assume that the user has entered the value 40 into the hoursWorkedTextBox control. The method converts the string "40" to the int value 40. The int value 40 is returned from the method and the = operator assigns it to the hoursWorked variable. The following Figure illustrates this process.



# Displaying Numeric Values

- Suppose an application has a decimal variable named **grossPay** and a Label control named `grossPayLabel`.
- To display the variable's value in the Label control, you must assign the value of the `grossPay` variable to the `grossPayLabel` control's `Text` property.
- If you try to use the following direct assignment statement, you will get an error because you cannot assign a numeric value to the `Text` property (only strings can be assigned).

```
grossPayLabel.Text = grossPay; ERROR!
```

- Therefore, to display the value of a numeric variable in a Label control, you must first convert the variable's value to a string.

# Displaying Numeric Values

- All variables have a ToString method that you can call to convert the variable's value to a string.
- The ToString method can be called using the following general format:

```
variableName.ToString();
```

- In the general format, variableName is the name of any variable.
- The expression returns the variable's value as a string.
- Here is a code sample that demonstrates:

```
decimal grossPay = 1550.0m;  
grossPayLabel.Text = grossPay.ToString();
```

# Displaying Numeric Values

- You must also convert a numeric variable to a string before passing it to the `MessageBox.Show` method.
- The following example shows how an `int` variable's value can be converted to a string and displayed in a message box:

```
int myNumber = 123;  
  
MessageBox.Show(myNumber.ToString());
```

# String Conversion with the + Operator

- The + operator has two uses: **string concatenation** and **numeric addition**.
- If you write an expression using the + operator and both operands are strings, the + operator concatenates the strings.
- If **both** operands are numbers of compatible types, then the + operator adds the two numbers.
- But what happens if one operand is a **string** and the other operand is a **number**?

The number will be implicitly converted to a string, and both operands will be concatenated.

# String Conversion with the + Operator

- **Example 1:** In the second statement, the string variable output is initialized with the string "**Your ID number is 1044**".

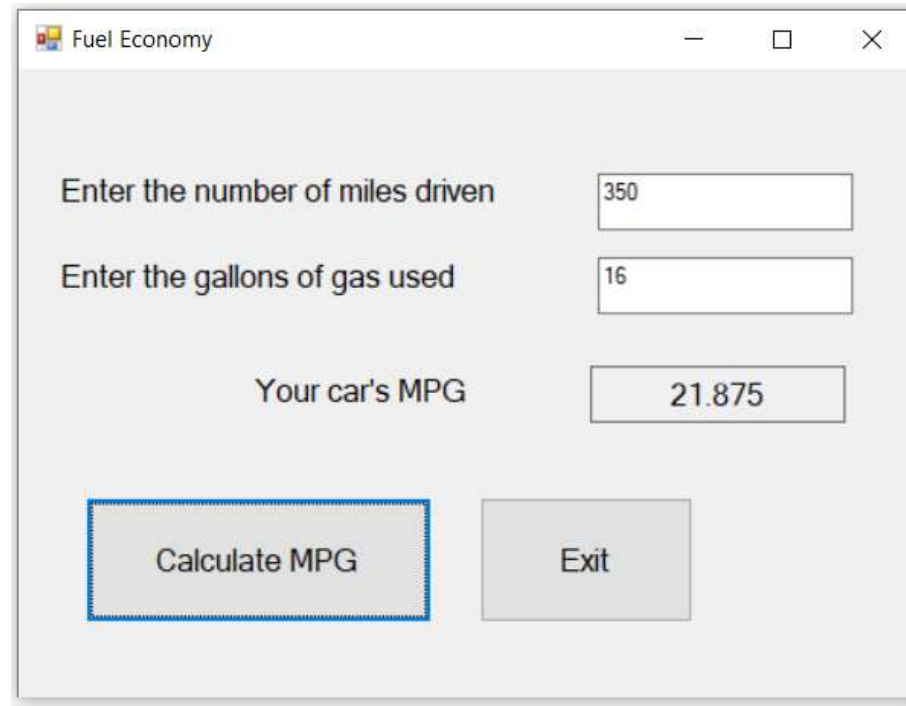
```
int idNumber = 1044;  
string output = "Your ID number is " + idNumber;
```

- **Example 2:** The second statement displays a message box showing the string "**Your test score is 88.5**".

```
double testScore = 88.5;  
MessageBox.Show("Your test score is " + testScore);
```

# Example 4

- **Calculating Fuel Economy:** Create a GUI application that lets the user enter the number of miles he or she has driven and the gallons of gas used. When the user clicks the “Calculate MPG” button, the application calculates the car’s MPG (e.g.,  $\text{MPG} = \text{Miles driven} \div \text{Gallons of gas used}$ ) and displays the result in the label control. When the user clicks the “Exit” button, the application’s form is closed.



The screenshot shows a window titled "Fuel Economy" with a light gray background. It contains two input fields for user data: "Enter the number of miles driven" with the value 350, and "Enter the gallons of gas used" with the value 16. Below these is a label "Your car's MPG" next to a display field showing the result 21.875. At the bottom are two buttons: "Calculate MPG" (highlighted with a blue dashed border) and "Exit".

| Input                            | Value  |
|----------------------------------|--------|
| Enter the number of miles driven | 350    |
| Enter the gallons of gas used    | 16     |
| Calculated MPG                   | 21.875 |

# Example 4

- 1) Set up the application's form as shown below.

The image shows a Java Swing window titled "Fuel Economy" with standard Windows-style window controls (minimize, maximize, close). The window contains the following components, each labeled with a text label and a red arrow pointing to it:

- milesPromptLabel**: Points to the text "Enter the number of miles driven".
- milesTextBox**: Points to the text input field for miles.
- gallonsPromptLabel**: Points to the text "Enter the gallons of gas used".
- gallonsTextBox**: Points to the text input field for gallons.
- outputDescriptionLabel**: Points to the text "Your car's MPG".
- mpgLabel**: Points to the output text field for MPG.
- calculateButton**: Points to the button labeled "Calculate MPG".
- exitButton**: Points to the button labeled "Exit".

# Example 4

2) Set up the properties of the controls as shown in the following table.

| Control Name           | Control Type | Property Settings                                                                                                                                                 |
|------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| milesPromptLabel       | Label        | <b>Text:</b> <i>Enter the number of miles driven:</i>                                                                                                             |
| gasPromptLabel         | Label        | <b>Text:</b> <i>Enter the gallons of gas used:</i>                                                                                                                |
| outputDescriptionLabel | Label        | <b>Text:</b> <i>Your car's MPG:</i>                                                                                                                               |
| milesTextBox           | TextBox      | No properties changed                                                                                                                                             |
| gallonsTextBox         | TextBox      | No properties changed                                                                                                                                             |
| mpgLabel               | Label        | <b>AutoSize:</b> False<br><b>BorderStyle:</b> FixedSingle<br><b>Text:</b> (The contents of the Text property have been erased.)<br><b>TextAlign:</b> MiddleCenter |
| calculateButton        | Button       | <b>Text:</b> <i>Calculate MPG</i>                                                                                                                                 |
| exitButton             | Button       | <b>Text:</b> <i>Exit</i>                                                                                                                                          |

# Example 4

- 3) Write the statement that you want to execute inside the event handler of Button controls, as shown on the right.

```
private void calculateButton_Click(object sender, EventArgs e)
{
    double miles;    // To hold miles driven
    double gallons;  // To hold gallons used
    double mpg;      // To hold MPG
                    //
    // Get the miles driven and assign it to the miles variable.
    miles = double.Parse(milesTextBox.Text);

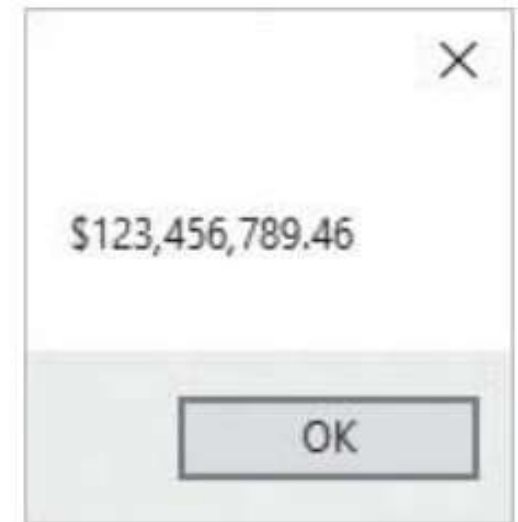
    // Get the gallons used and assign it to the gallons variable.
    gallons = double.Parse(gallonsTextBox.Text);
    mpg = miles / gallons; // Calculate MPG.

    // Display the MPG in the mpgLabel control.
    mpgLabel.Text = mpg.ToString();
}
1 reference
private void exitButton_Click(object sender, EventArgs e)
{
    this.Close(); // Close the form.
}
```

# Formatting Numbers with ToString Method

- The ToString method can optionally format a number to appear in a specific way.
- When you display large numbers, you usually want to format them with commas so they are easy to read.
- For example: 487,634,789.0 is easier to read than 487634789.0.
- Also, when you display amounts of money, you usually want to round them to two decimal places and display a currency symbol, such as a dollar sign (\$), as shown in the figure.

```
decimal amount = 123456789.45678m;  
MessageBox.Show(amount.ToString("c"));
```



# Formatting Numbers with ToString Method

- There are several other format strings that you can use with the ToString method, and each produces a different type of formatting. The following Table shows a few of them.

| Format String | Description                   |
|---------------|-------------------------------|
| "N" or "n"    | Number format                 |
| "F" or "f"    | Fixed-point scientific format |
| "E" or "e"    | Exponential scientific format |
| "C" or "c"    | Currency format               |
| "P" or "p"    | Percent format                |

# Formatting Numbers with ToString Method

## ■ Number Format:

- Displays numeric values with comma separators and a decimal point.
- By default, two digits display to the right of the decimal point. Negative values are displayed with a leading minus sign.
- An example is -2,345.67

## ■ Fixed-Point Format:

- Displays numeric values with no thousands separator and a decimal point.
- By default, two digits display to the right of the decimal point.
- Negative values are displayed with a leading minus (-) sign.
- An example is -2345.67

# Formatting Numbers with ToString Method

## ■ Exponential Format:

- Displays numeric values in scientific notation.
- The number is displayed with a single digit to the left of the decimal point.
- The letter e appears in front of the exponent, and the exponent has a leading + or - sign.
- By default, six digits display to the right of the decimal point, and a leading minus sign is used if the number is negative.
- An example is `-2.345670e+003`

# Formatting Numbers with ToString Method

## ■ Currency Format:

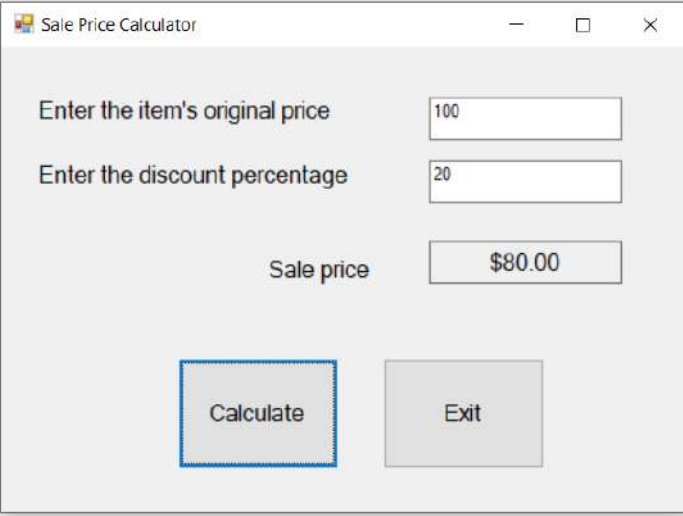
- Displays a leading currency symbol (such as \$), digits, comma separators, and a decimal point.
- By default, two digits display to the right of the decimal point. Negative values are surrounded by parentheses.
- An example of a negative value is (\$2,345.67)

## ■ Percent Format:

- Causes the number to be multiplied by 100 and displayed with a trailing space and % sign.
- By default, two digits display to the right of the decimal point.
- Negative values are displayed with a leading minus sign.
- For example, the number 0.125 would be formatted as 12.5 % and the number -0.2345 would be formatted as -23.45 %

# Example 5

- **Sale Price Calculator application with Currency Formatting:** Create an application that calculates the sale price of an item after the discount is subtracted. You should do the following:
  - Get the original price of the item.
  - Get the discount percentage. (For example, 20 is entered for 20 percent)
  - Divide the percentage amount by 100 to move the decimal point to the correct location.
  - Multiply the percentage by the original price. This is the amount of the discount.
  - Subtract the discount from the original price. This is the sale price.
  - Display the sale price.



The screenshot shows a window titled "Sale Price Calculator". It contains three input fields: "Enter the item's original price" with the value "100", "Enter the discount percentage" with the value "20", and "Sale price" with the value "\$80.00". At the bottom, there are two buttons: "Calculate" and "Exit".

# Example 5

- 1) Set up the application's form as shown below.

When the application runs, the user enters an item's original price into the originalPriceTextBox control and the discount percentage into the discountPercentageTextBox control.

When the user clicks the calculateButton control, the application calculates the item's sale price and displays the result in the salePriceLabel control. The exitButton control closes the application's form.

The image shows a screenshot of a Windows application window titled "Sale Price Calculator". The window contains a form with the following controls and labels:

- originalPricePromptLabel**: Points to the text "Enter the item's original price".
- originalPriceTextBox**: Points to the text input field for the original price.
- discPercentagePromptLabel**: Points to the text "Enter the discount percentage".
- discountPercentageTextBox**: Points to the text input field for the discount percentage.
- outputDescriptionLabel**: Points to the text "Sale price".
- salePriceLabel**: Points to the text box for displaying the calculated sale price.
- calculateButton**: Points to the "Calculate" button.
- exitButton**: Points to the "Exit" button.

# Example 5

2) Set up the properties of the controls as shown in the following table.

| Control Name              | Control Type | Property Settings                                                                                                                                                 |
|---------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| originalPricePromptLabel  | Label        | <b>Text:</b> <i>Enter the item's original price:</i>                                                                                                              |
| discPercentagePromptLabel | Label        | <b>Text:</b> <i>Enter the discount percentage:</i>                                                                                                                |
| outputDescriptionLabel    | Label        | <b>Text:</b> <i>Sale price:</i>                                                                                                                                   |
| originalPriceTextBox      | TextBox      | No properties changed                                                                                                                                             |
| discountPercentageTextBox | TextBox      | No properties changed                                                                                                                                             |
| salePriceLabel            | Label        | <b>AutoSize:</b> False<br><b>BorderStyle:</b> FixedSingle<br><b>Text:</b> (The contents of the Text property have been erased.)<br><b>TextAlign:</b> MiddleCenter |
| calculateButton           | Button       | <b>Text:</b> <i>Calculate Sale Price</i>                                                                                                                          |
| exitButton                | Button       | <b>Text:</b> <i>Exit</i>                                                                                                                                          |

# Example 5

- 3) Write the statement that you want to execute inside the event handler of Button controls, as shown on the right.

```
private void calculateButton_Click(object sender, EventArgs e)
{
    decimal originalPrice; // The item's original price
    decimal discountPercentage; // The discount percentage
    decimal discountAmount; // The amount of discount
    decimal salePrice; // The item's sale price
    // Get the item's original price.
    originalPrice = decimal.Parse(originalPriceTextBox.Text);
    // Get the discount percentage.
    discountPercentage = decimal.Parse(discountPercentageTextBox.Text);
    // Move the percentage's decimal point left two spaces.
    discountPercentage = discountPercentage / 100;
    // Calculate the amount of the discount.
    discountAmount = originalPrice * discountPercentage;
    // Calculate the sale price.
    salePrice = originalPrice - discountAmount;

    // Display the sale price.
    salePriceLabel.Text = salePrice.ToString("c");
}

private void exitButton_Click(object sender, EventArgs e)
{
    // Close the form.
    this.Close();
}
```

# Exception Handling

- An exception is an unexpected error that occurs while a program is running, causing the program to halt if the error is not properly dealt with.
- For example, suppose the user has entered a value into a TextBox, and that value is expected to be a number. The program uses one of the Parse methods to convert the control's Text property to a numeric data type, but the string contains invalid characters and it cannot be converted. The Parse method cannot continue, so an exception occurs.
- C# allows you to write code that responds to exceptions when they are thrown and prevents the program from abruptly crashing. Such code is called an exception handler, and is written with the try-catch statement. There are several ways to write a try-catch statement, but the general format that is the simplest variation is shown on the right.

```
try
{
    Statement(s);
}
catch
{
    Statement(s);
}
```

# Exception Handling

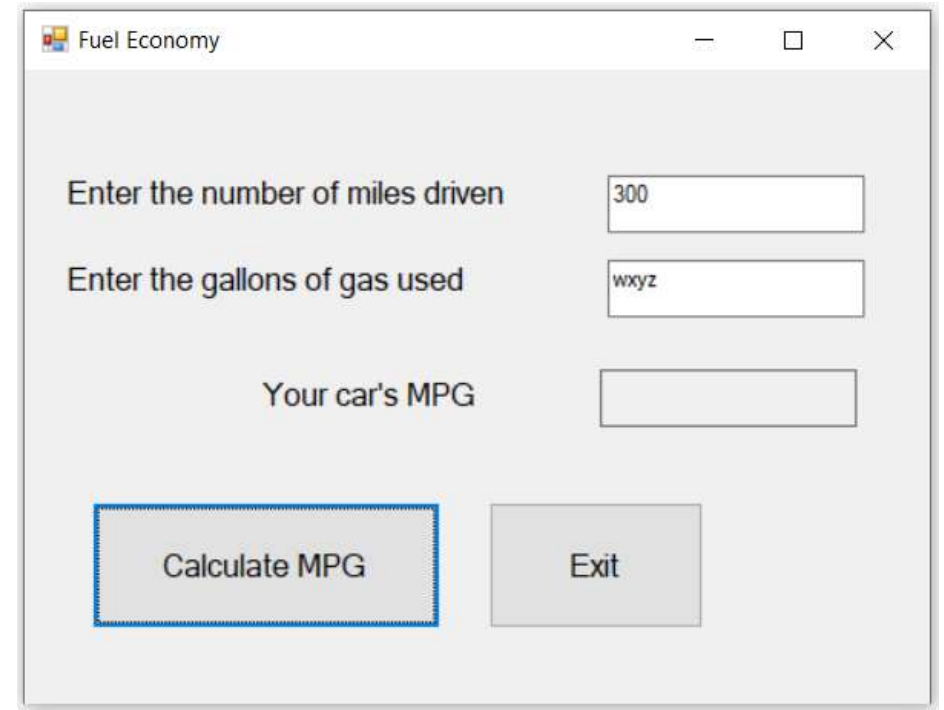
- When an exception is thrown, an object known as an exception object is created in memory.
- The exception object has various properties that contain data about the exception.
- When we write a catch clause, we can optionally assign a name to the exception object, as shown here:

```
catch(Exception ex)
```

- The expression that appears inside the parentheses specifies that we are assigning the name `ex` to the exception object.

# Exception Handling–Fuel Economy Application

- In the Calculating Fuel Economy application in Example 4, suppose that the user enters 300 for the number of miles driven and then enter a nonnumeric sequence of characters (e.g., wxyz) for the gallons of gas used, as shown on the right.
- When the user clicks the Calculate MPG button, an exception is thrown because the invalid string entered for the gallons of gas cannot be converted to a double.
- As a result, the application stops running and Visual Studio goes into a special mode known as break mode. To prevent this issue, the code should be written inside a try-catch statement.



The screenshot shows a Windows application window titled "Fuel Economy". It contains three input fields and two buttons. The first input field is labeled "Enter the number of miles driven" and contains the value "300". The second input field is labeled "Enter the gallons of gas used" and contains the text "wxyz". The third input field is labeled "Your car's MPG" and is empty. Below the input fields are two buttons: "Calculate MPG" and "Exit". The "Calculate MPG" button is highlighted with a blue dashed border.

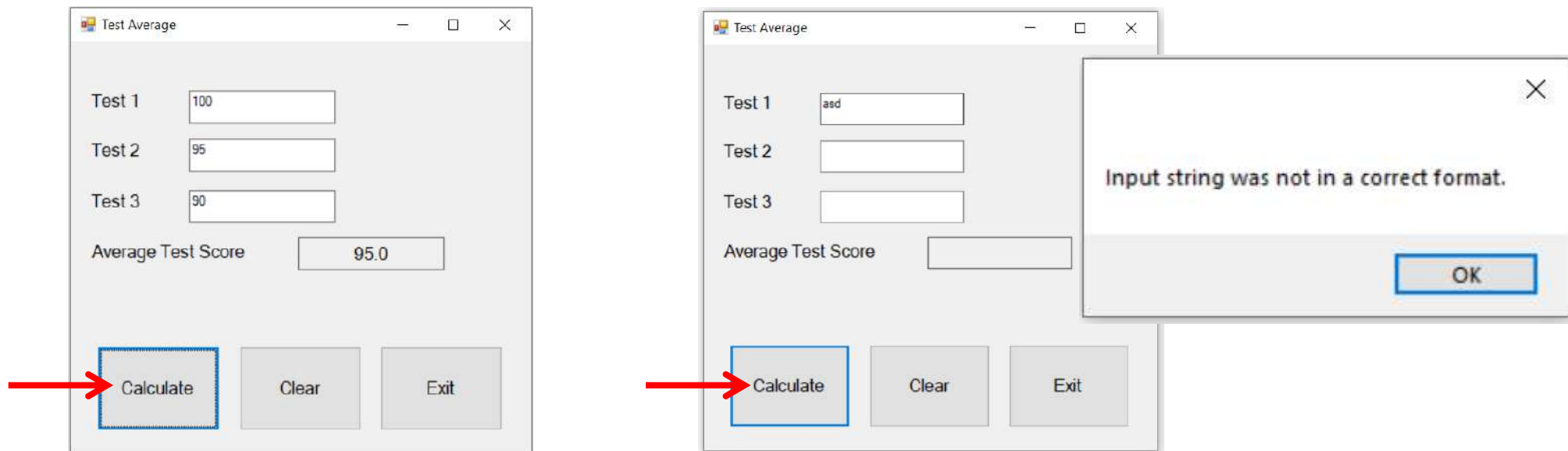
# Exception Handling–Fuel Economy Application

- To prevent this issue, the code should be written inside a try-catch statement, as shown on the right.

```
private void calculateButton_Click(object sender, EventArgs e)
{
    try
    {
        double miles;    // To hold miles driven
        double gallons;  // To hold gallons used
        double mpg;      // To hold MPG
        // Get the miles driven and assign it to the miles variable.
        miles = double.Parse(milesTextBox.Text);
        // Get the gallons used and assign it to the gallons variable.
        gallons = double.Parse(gallonsTextBox.Text);
        mpg = miles / gallons; // Calculate MPG.
        // Display the MPG in the mpgLabel control.
        mpgLabel.Text = mpg.ToString();
    }
    catch
    {
        // Display an error message
        MessageBox.Show("Invalid data was entered");
    }
}
```

# Example 6

- **Test Average Application with Exception Handling:** Create an application that calculates the average of three test scores. When the user enters valid numeric values into the TextBoxes and click the Calculate button, a test average should be displayed. When the user clicks the Clear button, the contents of the TextBoxes and the outputLabel control are cleared. When the user enters a nonnumeric value for test 1 and clicks the Calculate button, an exception is thrown and a message box should be displayed to indicate an invalid data. The “Exit” button closes the application’s form.



# Example 6

1) Set up the application's form as shown below.

The image shows a screenshot of a Windows-style application window titled "Test Average". The window contains a form with three input fields for test scores, a label for the average score, and three buttons at the bottom. Red arrows point from external labels to the corresponding UI elements:

- test1PromptLabel** points to the "Test 1" label.
- test2PromptLabel** points to the "Test 2" label.
- test3PromptLabel** points to the "Test 3" label.
- outputDescriptionLabel** points to the "Average Test Score" label.
- test1TextBox** points to the first text input box.
- test2TextBox** points to the second text input box.
- test3TextBox** points to the third text input box.
- averageLabel** points to the text input box for the average score.
- calculateButton** points to the "Calculate" button.
- clearButton** points to the "Clear" button.
- exitButton** points to the "Exit" button.

# Example 6

2) Set up the properties of the controls as shown in the following table.

| Control Name           | Control Type | Property Settings                                                                                                                                                 |
|------------------------|--------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| test1PromptLabel       | Label        | <b>Text:</b> <i>Test 1:</i>                                                                                                                                       |
| test2PromptLabel       | Label        | <b>Text:</b> <i>Test 2:</i>                                                                                                                                       |
| test3PromptLabel       | Label        | <b>Text:</b> <i>Test 3:</i>                                                                                                                                       |
| outputDescriptionLabel | Label        | <b>Text:</b> <i>Average Test Score:</i>                                                                                                                           |
| test1TextBox           | TextBox      | No properties changed                                                                                                                                             |
| test2TextBox           | TextBox      | No properties changed                                                                                                                                             |
| test3TextBox           | TextBox      | No properties changed                                                                                                                                             |
| averageLabel           | Label        | <b>AutoSize:</b> False<br><b>BorderStyle:</b> FixedSingle<br><b>Text:</b> (The contents of the Text property have been erased.)<br><b>TextAlign:</b> MiddleCenter |
| calculateButton        | Button       | <b>Text:</b> <i>Calculate Sale Price</i>                                                                                                                          |
| clearButton            | Button       | <b>Text:</b> <i>Clear</i>                                                                                                                                         |
| exitButton             | Button       | <b>Text:</b> <i>Exit</i>                                                                                                                                          |

# Example 6

- 3) Write the statements that compute and display the average of the scores inside the calculateButton's event handler, as shown on the right.

```
private void calculateButton_Click(object sender, EventArgs e)
{
    try
    {
        double test1;    // To hold test score #1
        double test2;    // To hold test score #2
        double test3;    // To hold test score #3
        double average;  // To hold the average test score

        // Get the three test scores.
        test1 = double.Parse(test1TextBox.Text);
        test2 = double.Parse(test2TextBox.Text);
        test3 = double.Parse(test3TextBox.Text);
        // Calculate the average test score.
        average = (test1 + test2 + test3) / 3.0;

        // Display the average test score, with
        // the output rounded to 1 decimal point.
        averageLabel.Text = average.ToString("n1");
    }
    catch (Exception ex)
    {
        // Display the default error message.
        MessageBox.Show(ex.Message);
    }
}
```

# Example 6

- 4) Write the statements to clear the TextBoxes and the Label inside the clearButton event handler and the statements that close the form application inside the exitButton's event handler, as shown on the right.
- 5) Run the application to view the output.

```
private void clearButton_Click(object sender, EventArgs e)
{
    // Clear the input and output controls.
    test1TextBox.Text = "";
    test2TextBox.Text = "";
    test3TextBox.Text = "";
    averageLabel.Text = "";
}

1 reference
private void exitButton_Click(object sender, EventArgs e)
{
    // Close the form.
    this.Close();
}
```

# Declaring Variables as Fields

- All the variables with which we have worked have been local variables.
- A local variable is declared inside a method and is visible only to statements in that method.
- Another type of variable is a field, which is a variable that is declared inside a class but not inside of any method.
- A field's scope is the entire class, so when you declare a field, all the methods in the class can access the variable.
- Typically, fields are declared at the top of a class declaration, before any methods.

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Threading.Tasks;
using System.Windows.Forms;
namespace MyFirstProject
{
    2 references
    public partial class Form1 : Form
    {
        ← Insert field declarations here
        0 references
        public Form1()
        {
            InitializeComponent();
        }
    }
}
```

# Field declarations

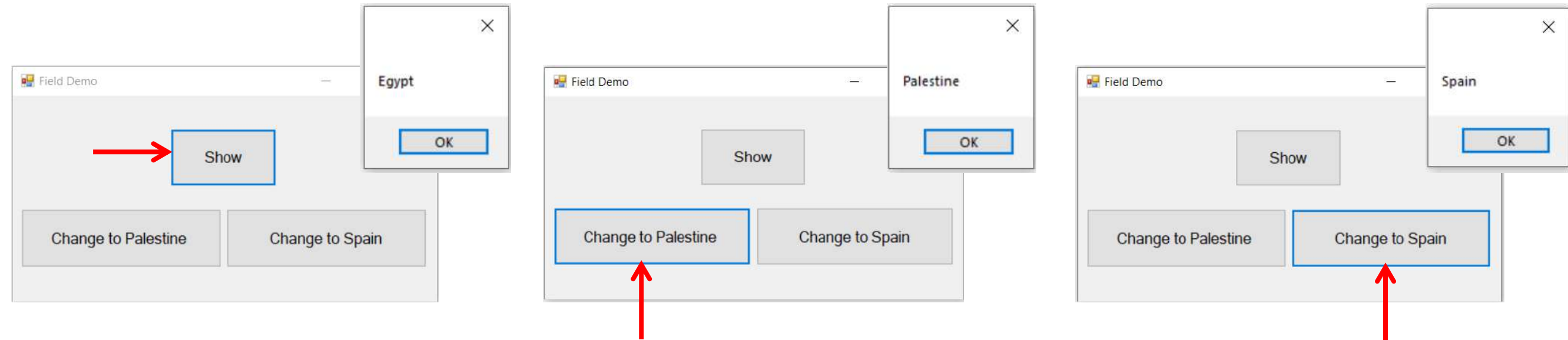
- Assume that the following statement appears inside a class declaration but not inside any methods.

```
private int number = 0;
```

- The keyword `private` is the access modifier.
- An access modifier specifies how a class member can be accessed by code outside the class.
- When you use the `private` access modifier in a field declaration, the field cannot be accessed by code outside the class.
- It can be accessed only by the methods that are inside the class.
- If you don't write an access modifier in a field declaration, C# will automatically make the field `private`.

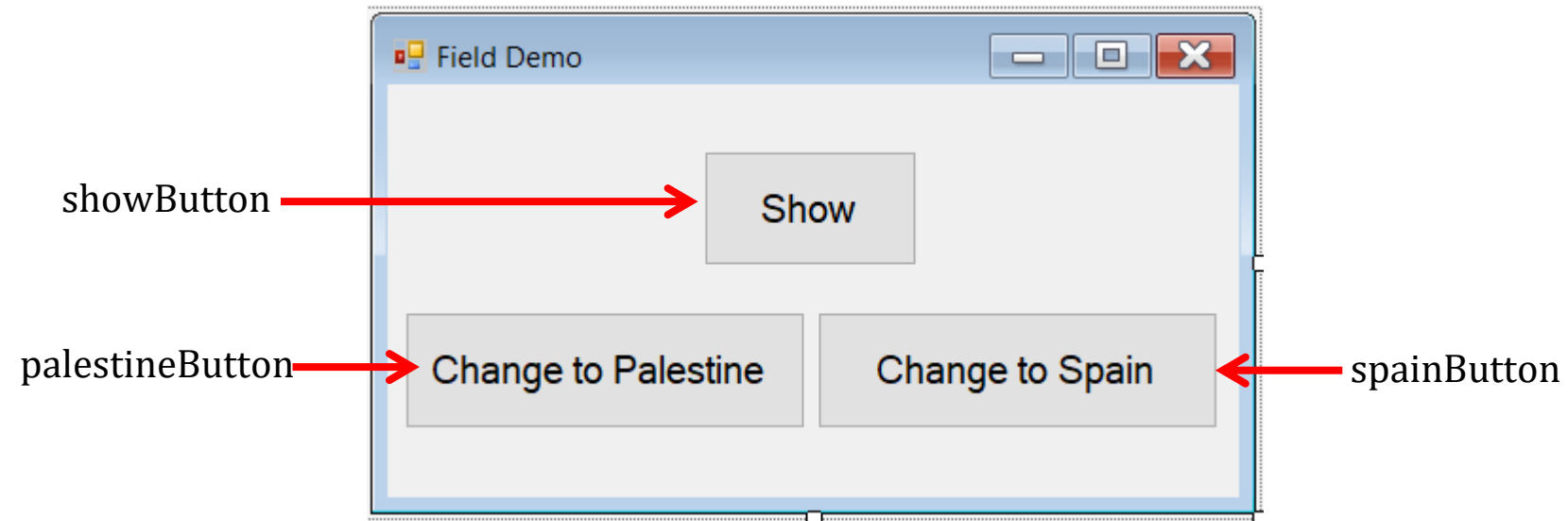
# Example 7

- **Field Application:** Create an application where all event handlers in the Form class have access to the name variable. When the user clicks the Show button, a message box appears displaying Egypt, which is the name field's initial value. When the user clicks the Change to Palestine button and then clicks the Show button, a message box appears displaying Palestine. When the user clicks the Change to Spain button and then clicks the Show button, a message box appears displaying Spain.



# Example 7

1) Set up the application's form as shown below.



# Example 7

- 2) Write the statements shown on the right inside the event handlers and run the application to view the output.

```
public partial class Form13 : Form
{
    // Declare a private field to hold a name.
    private string name = "Egypt";
    1 reference
    public Form13()
    {
        InitializeComponent();
    }

    1 reference
    private void showButton_Click(object sender, EventArgs e)
    {
        MessageBox.Show(name);
    }

    1 reference
    private void palestineButton_Click(object sender, EventArgs e)
    {
        name = "Palestine";
    }

    1 reference
    private void SpainButton_Click(object sender, EventArgs e)
    {
        name = "Spain";
    }
}
```

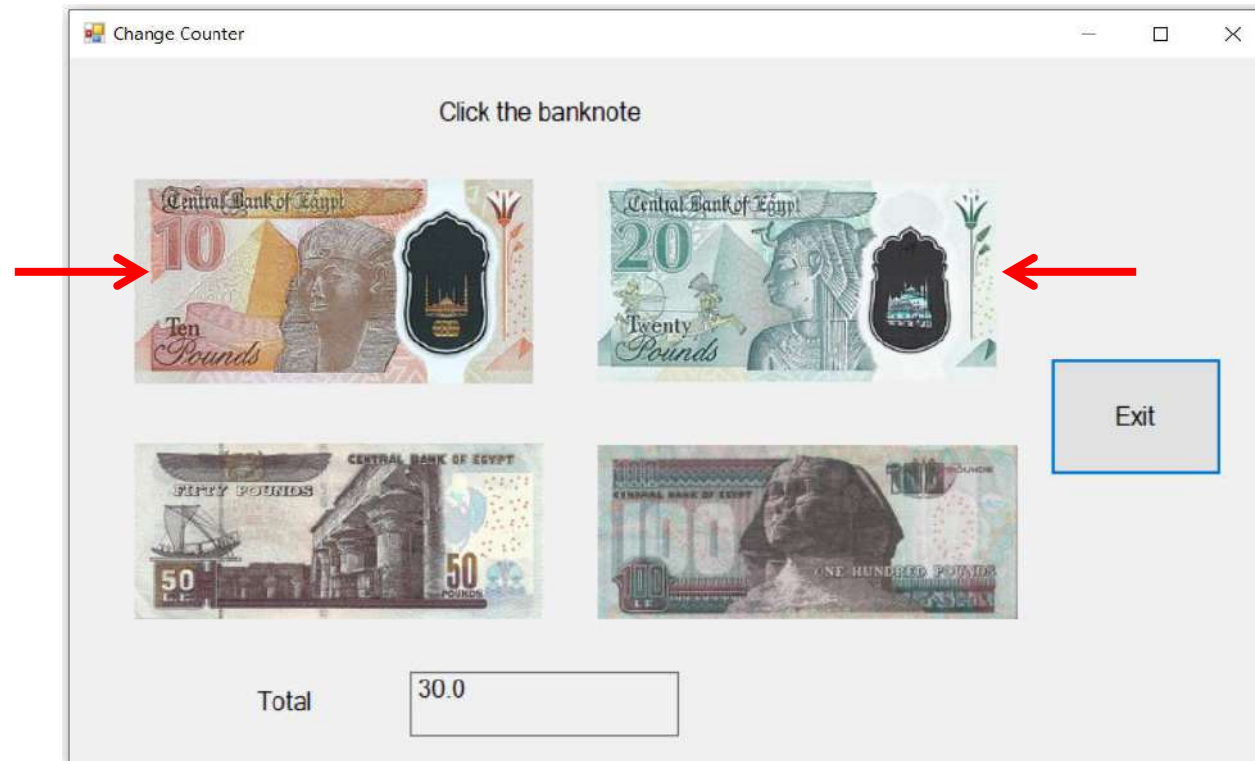
# Constant Field

- A field that cannot be changed by any statement in the class.
- An error will occur if the compiler finds a statement that tries to change the value of a constant field.

```
private const decimal interestRate = 0.075m;
```

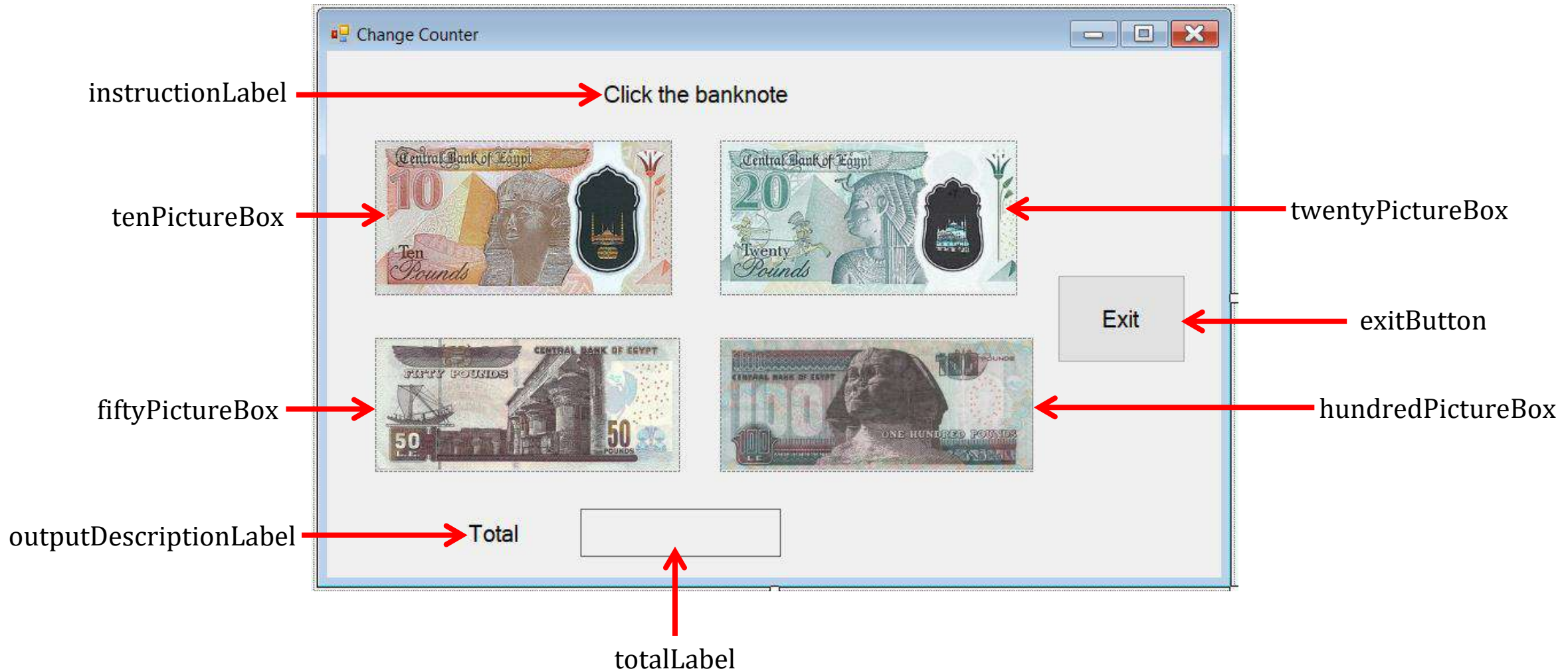
# Example 8

- **Change Counter application:** Create an application displays images of four banknotes, having the values 10 pounds, 20 pounds, 50 pounds, and 100 pounds. Each time the user clicks on a banknote image, the value of that banknote is added to a total, and the total is displayed. The “Exit” button closes the application’s form.



# Example 8

1) Set up the application's form as shown below.



# Example 8

2) Set up the properties of the controls as shown in the following table.

| Control Name           | Control Type | Property Settings                                                                                                                                                                                                                       |
|------------------------|--------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| instructionLabel       | Label        | <b>Font:</b> Microsoft Sans Serif (Style: Regular, Size: 12 point)<br><b>Text:</b> Click the banknotes                                                                                                                                  |
| tenPictureBox          | PictureBox   | <b>Image:</b> Select and import 10 pounds image<br><b>SizeMode:</b> Autosize                                                                                                                                                            |
| twentyPictureBox       | PictureBox   | <b>Image:</b> Select and import 20 pounds image<br><b>SizeMode:</b> Autosize                                                                                                                                                            |
| fiftyPictureBox        | PictureBox   | <b>Image:</b> Select and import 50 pounds image<br><b>SizeMode:</b> Autosize                                                                                                                                                            |
| hundredPictureBox      | PictureBox   | <b>Image:</b> Select and import 100 pounds image<br><b>SizeMode:</b> Autosize                                                                                                                                                           |
| outputDescriptionLabel | Label        | <b>Font:</b> Microsoft Sans Serif (Style: Regular, Size: 12 point)<br><b>Text:</b> Total                                                                                                                                                |
| totalLabel             | Label        | <b>AutoSize:</b> False<br><b>BorderStyle:</b> FixedSingle<br><b>Font:</b> Microsoft Sans Serif (Style: Regular, Size: 12 point)<br><b>Text:</b> (The contents of the Text property have been erased.)<br><b>TextAlign:</b> MiddleCenter |
| exitButton             | Button       | <b>Text:</b> Exit                                                                                                                                                                                                                       |

# Example 8

- 3) Write the statements shown on the right inside the PictureBoxes event handlers and run the application to view the output.

```
public partial class Form14 : Form
{
    // Constant fields
    const decimal TEN_POUNDS_VALUE = 10.0m;
    const decimal TWENTY_POUNDS_VALUE = 20.0m;
    const decimal FIFTY_POUNDS_VALUE = 50.0m;
    const decimal HUNDRED_POUNDS_VALUE = 100.0m;
    // Field variable to hold the total, initialized with 0.
    private decimal total = 0m;

    public Form14()
    {
        InitializeComponent();
    }

    private void tenPictureBox_Click(object sender, EventArgs e)
    {
        total += TEN_POUNDS_VALUE; // Add the value of 10 pounds to the total.
        totalLabel.Text = total.ToString(); // Display the total.
    }

    private void twentyPictureBox_Click(object sender, EventArgs e)
    {
        total += TWENTY_POUNDS_VALUE; // Add the value of 20 pounds to the total.
        totalLabel.Text = total.ToString(); // Display the total.
    }

    private void fiftyPictureBox_Click(object sender, EventArgs e)
    {
        total += FIFTY_POUNDS_VALUE; // Add the value of 50 pounds to the total.
        totalLabel.Text = total.ToString(); // Display the total.
    }

    private void hundredPictureBox_Click(object sender, EventArgs e)
    {
        total += HUNDRED_POUNDS_VALUE; // Add the value of 100 pounds to the total.
        totalLabel.Text = total.ToString(); // Display the total.
    }

    private void exitButton_Click(object sender, EventArgs e)
    {
        this.Close(); // Close the form.
    }
}
```

# Math Class

| Method           | Description                                                                                                         |
|------------------|---------------------------------------------------------------------------------------------------------------------|
| Math.Abs(x)      | Returns the absolute value of x.                                                                                    |
| Math.Sqrt(x)     | Returns the square root of x (a double). The value that is returned is a double.                                    |
| Math.Floor(x)    | Returns the greatest integer that is less than or equal to x (a decimal or a double).                               |
| Math.Ceiling(x)  | Returns the least integer that is greater than or equal to x (a decimal or a double).                               |
| Math.Round(x)    | Returns the value of x (a double or a decimal) rounded to the nearest integer.                                      |
| Math.Truncate(x) | Returns the integer part of x (a double or a decimal).                                                              |
| Math.Max(x, y)   | Returns the greater of the two values x and y.                                                                      |
| Math.Min(x, y)   | Returns the lesser of the two values x and y.                                                                       |
| Math.Pow(x, y)   | Returns the value of x (a double) raised to the power of y (also a double). The value that is returned is a double. |

# Controlling a Form's Tab Order

- When an application is running and a form is displayed, one of the form's controls always has the focus.
- The control having the focus is the one that receives the user's keyboard input.
- For example, when a TextBox control has the focus, it receives the characters that the user enters on the keyboard.
- When a button has the focus, pressing the Enter key executes the button's Click event handler.
- You can tell which control has the focus by looking at the form at run time.
- When a TextBox control has the focus, a blinking text cursor appears inside it, or the text inside the TextBox control might appear highlighted.
- When a button has the focus, a thin dotted line usually appears around the control.

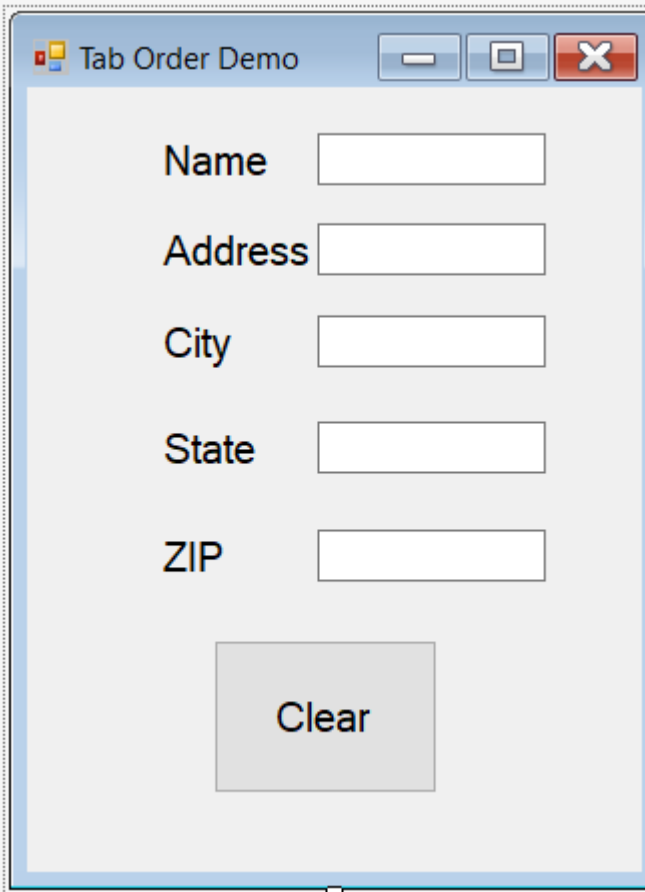
# Controlling a Form's Tab Order

- When an application is running, pressing the Tab key changes the focus from one control to another.
- The order in which controls receive the focus is called the tab order.
- When you place controls on a form in Visual C#, the tab order is in the same sequence in which you created the controls.
- You can modify the tab order by changing a control's **TabIndex** property.
- The **TabIndex** property contains a numeric value, which indicates the control's position in the tab order.
- When you create a control, Visual C# automatically assigns a value to its **TabIndex** property.
- The first control you create on a form has a **TabIndex** of 0, the second has a **TabIndex** of 1, and so forth.

# Controlling a Form's Tab Order

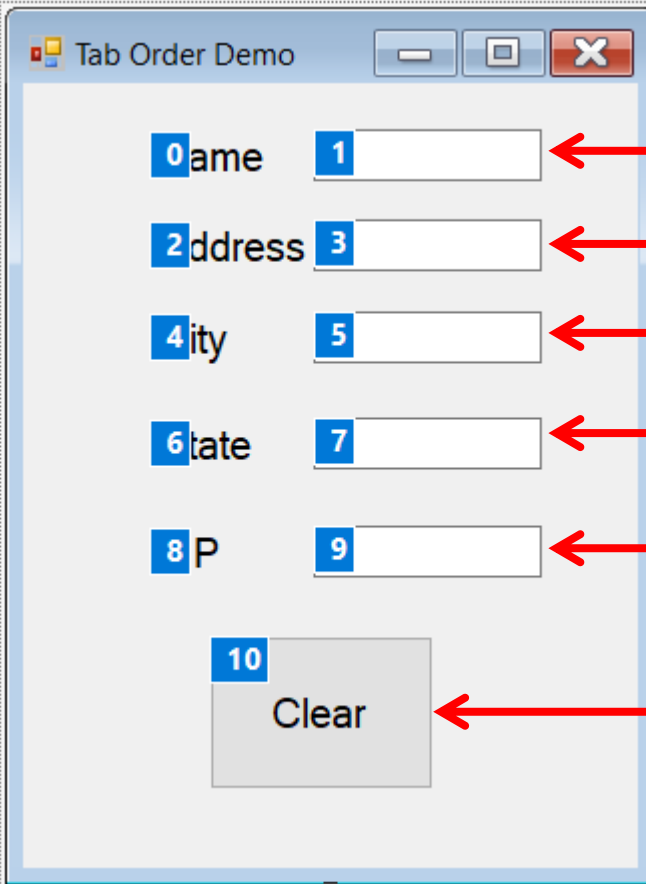
- The control with a TabIndex of 0 is the first control in the tab order. The next control in the tab order is the one with a TabIndex of 1. The tab order continues in this sequence.
- Click View on the Visual Studio menu bar and then click Tab Order, this causes the form to be displayed in tab order selection mode.

Form in the normal view



A screenshot of a Windows form titled "Tab Order Demo". The form contains five text boxes labeled "Name", "Address", "City", "State", and "ZIP" arranged vertically. Below them is a "Clear" button. The form is in its normal view, with no tab order indicators.

Form in tab order selection mode



A screenshot of the same "Tab Order Demo" form in tab order selection mode. Each control has a blue square with a white number indicating its TabIndex. Red arrows point from labels on the right to the corresponding controls. The labels are: "nameTextBox" (points to TabIndex 1), "addressTextBox" (points to TabIndex 3), "cityTextBox" (points to TabIndex 5), "stateTextBox" (points to TabIndex 7), "zipTextBox" (points to TabIndex 9), and "clearButton" (points to TabIndex 10). The TabIndex for "Name" is 0, "Address" is 2, "City" is 4, "State" is 6, and "ZIP" is 8.

# Changing the Focus with the Focus Method

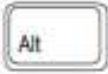
- Focus method is used to move the focus to a control. The method's general syntax is:

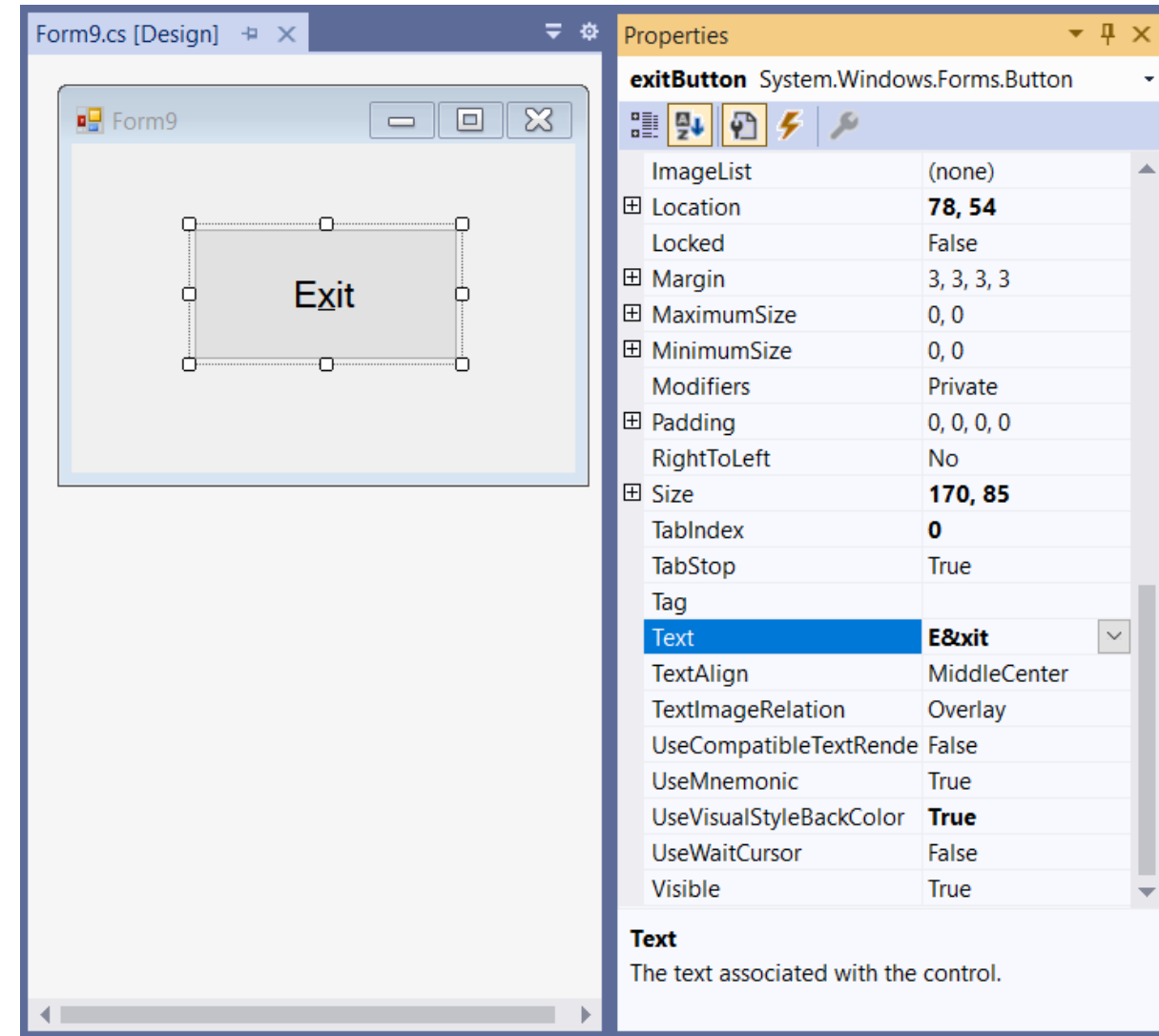
```
ControlName.Focus();
```

- **Example:** The purpose of the Clear button is to clear any input that the user has entered and reset the form so it is ready to accept a new set of input. When the Clear button is clicked, the TextBox controls should be cleared and the focus should return to the nameTextBox control.

```
1 private void clearButton_Click(object sender, EventArgs e)
2 {
3     // Clear the TextBox controls.
4     nameTextBox.Text = "";
5     addressTextBox.Text = "";
6     cityTextBox.Text = "";
7     stateTextBox.Text = "";
8     zipTextBox.Text = "";
9
10    // Set the focus to nameTextBox.
11    nameTextBox.Focus();
12 }
```

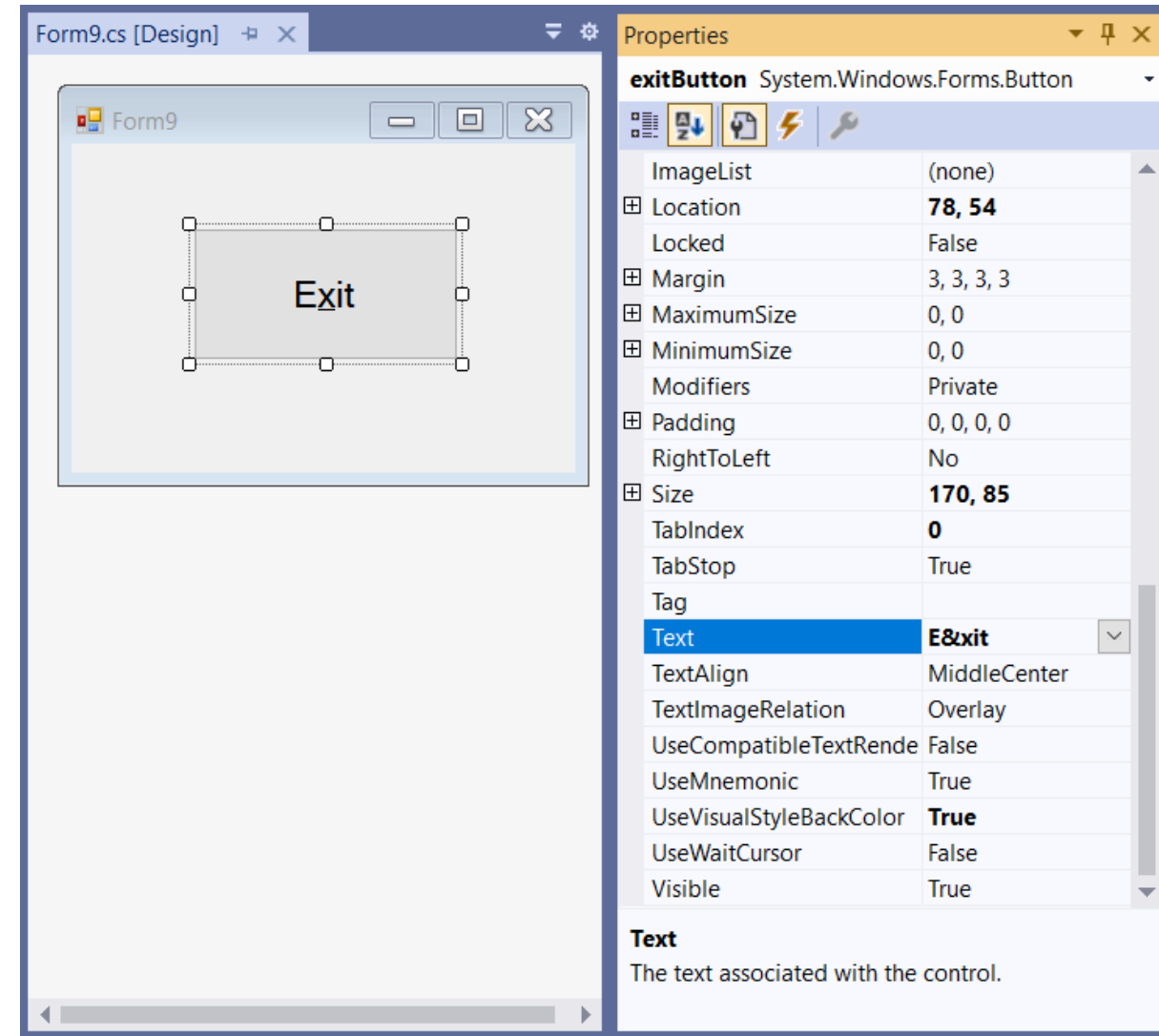
# Assigning Keyboard Access Keys to Buttons

- An access key, is a key that is pressed in combination with the **Alt** key to access quickly a control such as a button.
- You assign an access key to a button through its Text property.
- For example, assume an application has a button whose Text property is set to Exit.
- You wish to assign the access key  +  to the button so the user may trigger the button's Click event by pressing  +  on the keyboard.



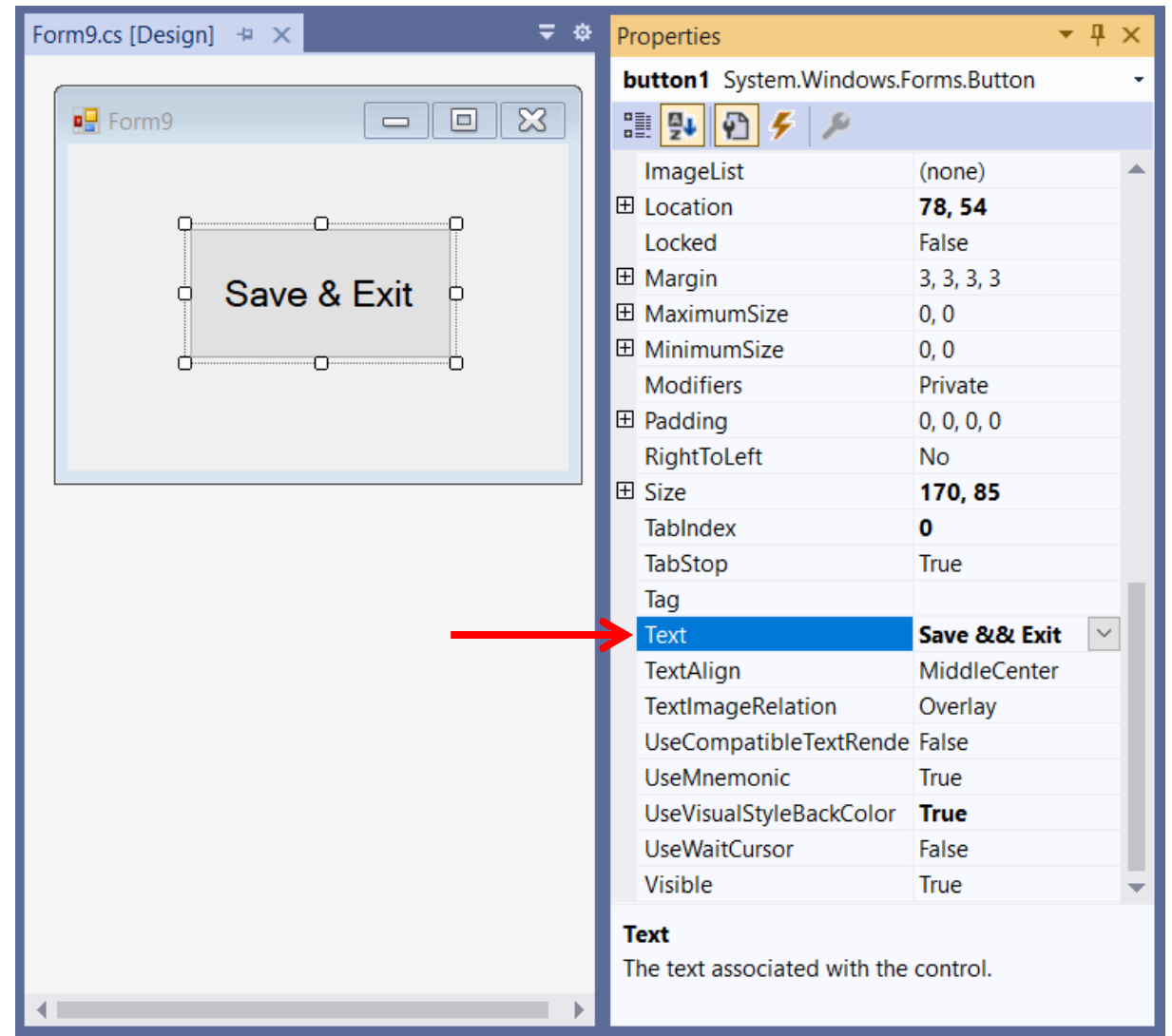
# Assigning Keyboard Access Keys to Buttons

- To make the assignment, place an ampersand (&) before the letter x in the button's Text property E&xit, as shown in the figure.
- Although the ampersand is part of the Button control's Text property, it is not displayed on the button.
- With the ampersand in front of the letter x, the letter will appear underlined as shown in the figure.
- This indicates that the button may be clicked by pressing  +  on the keyboard.



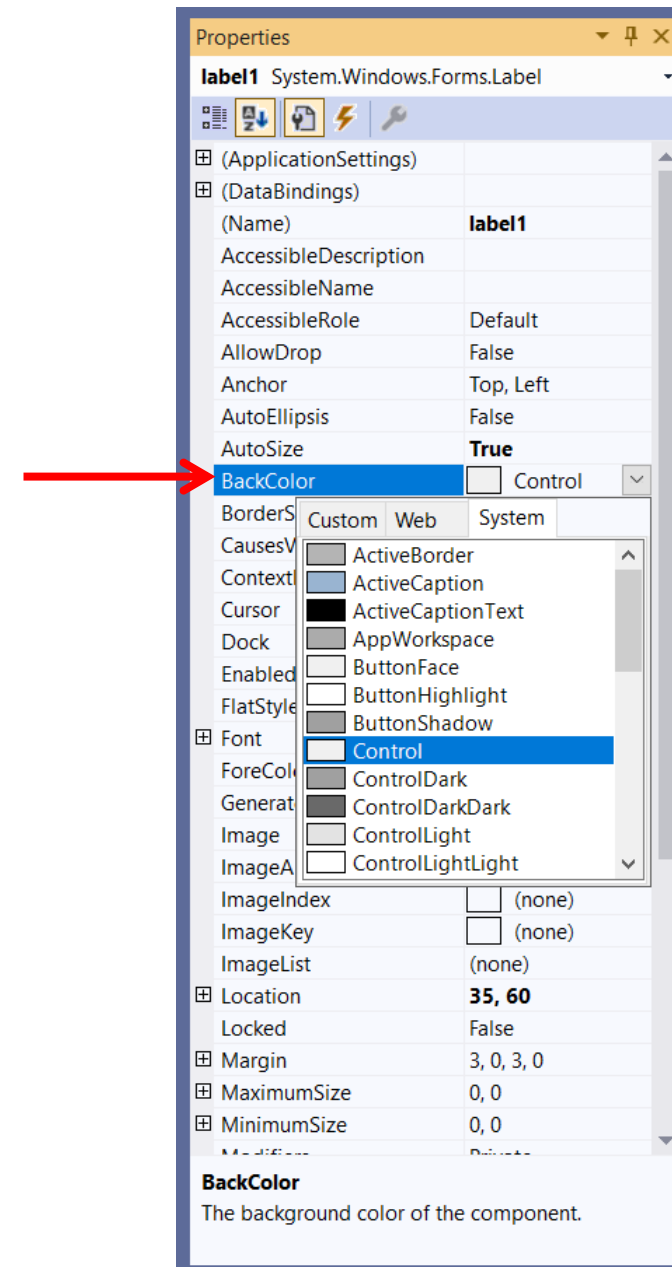
# Displaying the & Character on a Button

- If you want to display an ampersand character on a button, use two ampersands (&&) in the Text property.
- Using two ampersands causes a single ampersand to display and does not define an access key.
- For example, if a button's Text property is set to **Save && Exit**, the button will appear as shown in the figure.



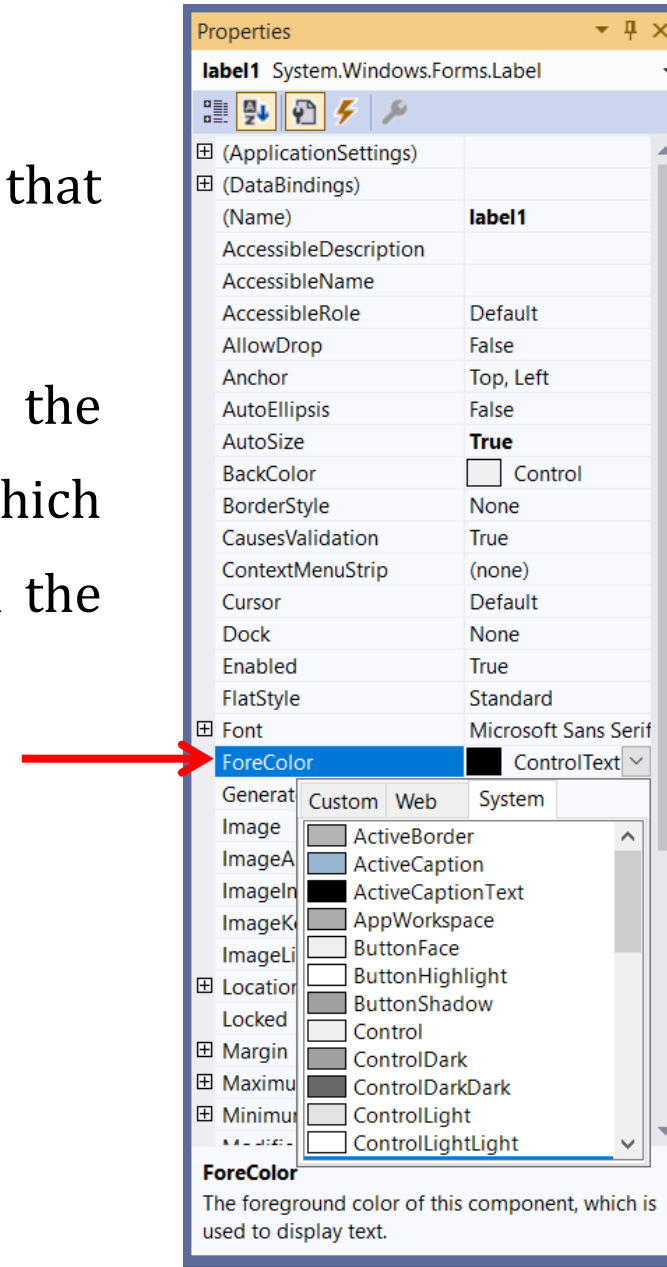
# BackColor Property

- Forms and most controls have a BackColor property that allows you to change the object's background color.
- When you select an object's BackColor property in the Properties window, a down-arrow (▼) button appears, which displays a drop-down list of available colors when clicked, as shown in the figure.



# ForeColor Property

- Controls that display text have a ForeColor property that allows you to change the color of the text.
- When you select a control's ForeColor property in the Properties window, a down-arrow (▼) button appears, which displays the drop-down list of available colors shown in the figure when clicked.



# Setting Colors in Code

- In addition to using the Properties window, you can also set the values of the **BackColor** and **ForeColor** properties with code. The following are a few of the values:

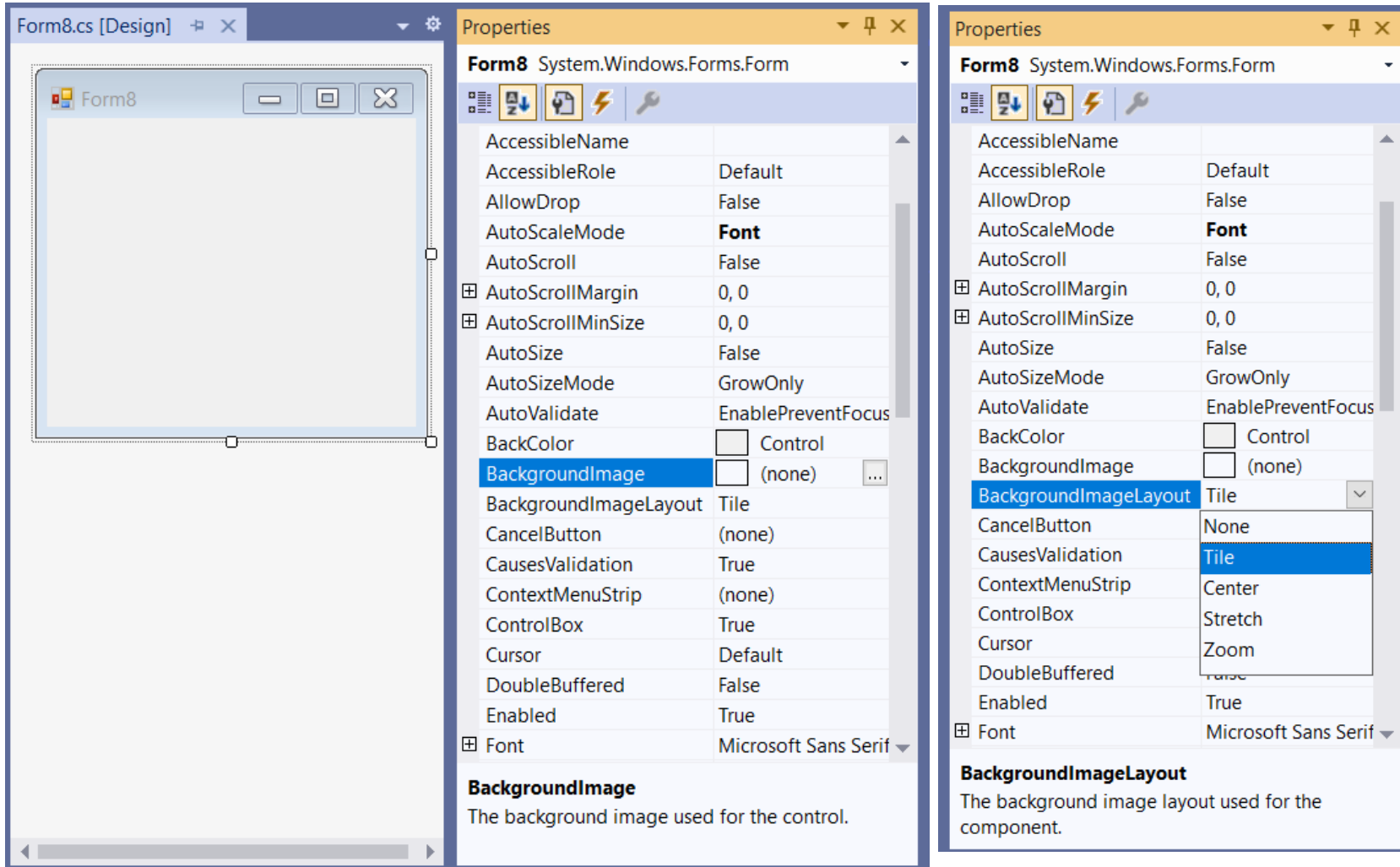
- Color.Black
- Color.Blue
- Color.Cyan
- Color.Green
- Color.Magenta
- Color.Red
- Color.White
- Color.Yellow

- For example, assume an application has a Label control named `messageLabel`. The following code sets the label's background color to black and foreground color to yellow:

```
messageLabel.BackColor = Color.Black;  
messageLabel.ForeColor = Color.Yellow;
```

# Background Images for Forms

- Forms have a property named **BackgroundImage** that allows you to import and display an image on the form.
- Forms also have a property named **BackgroundImageLayout** that specifies how the background image is to be displayed.

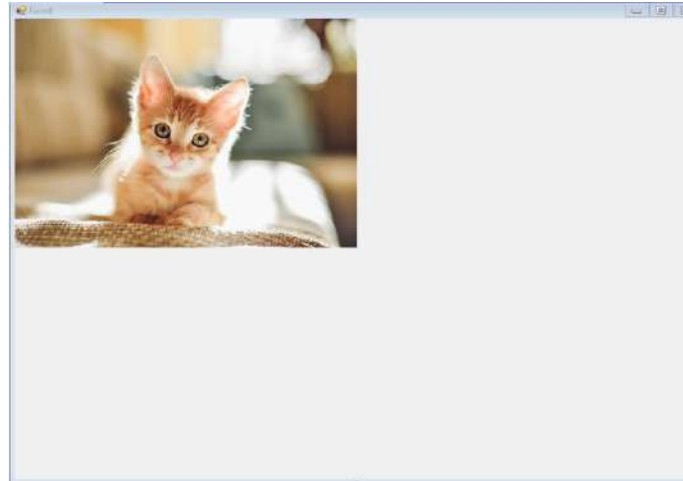


# Background Images for Forms

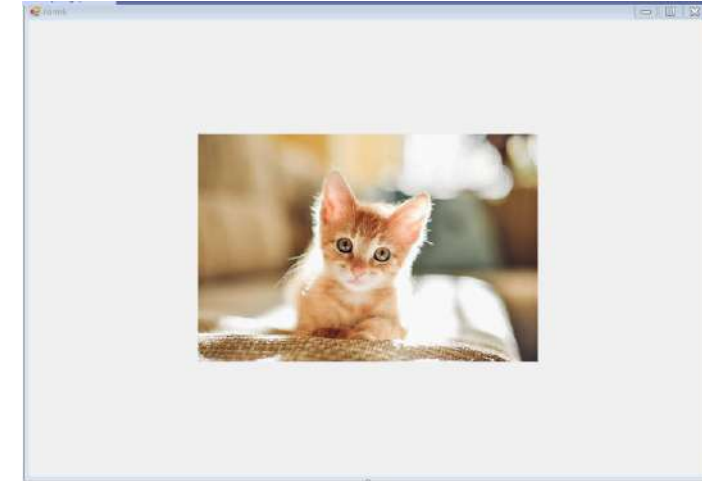
■ The **BackgroundImageLayout** property can be set to one of the following values:

- **None:** The image is positioned in the upper-left corner of the form. If the image is too big to fit in the form, it is clipped.
- **Tile:** This is the default value. The image is tiled (repeatedly displayed) across the form.
- **Center:** The image is centered in the form without being resized.

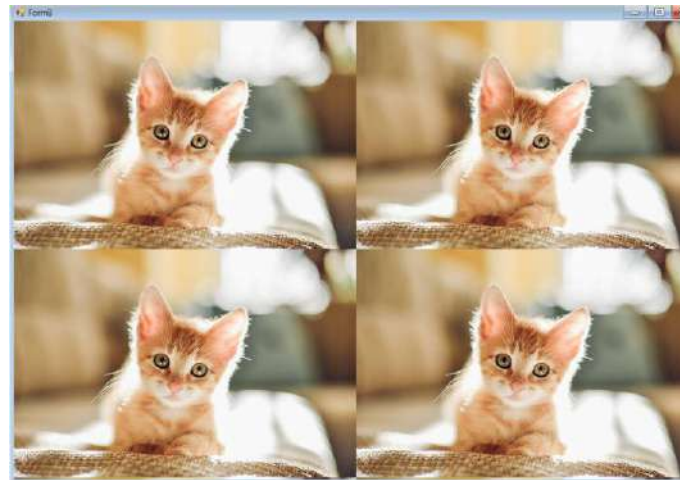
None



Center



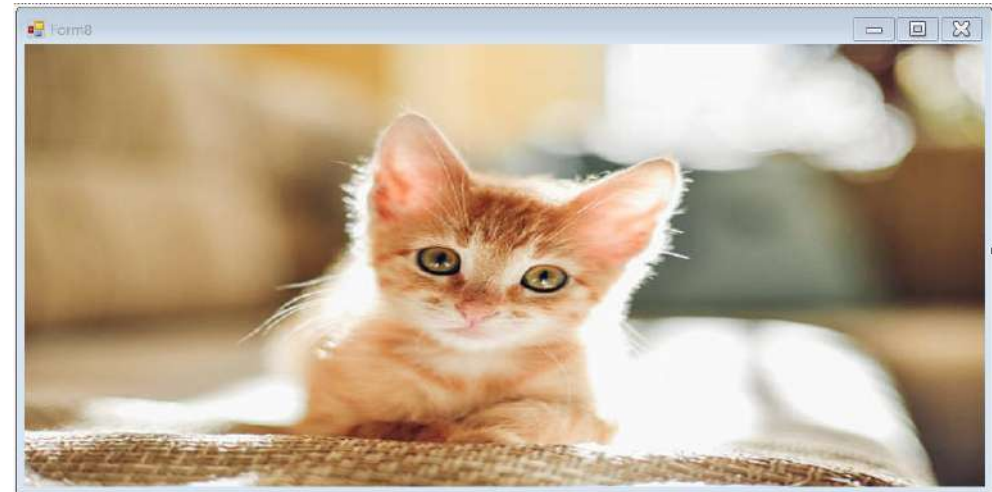
Tile



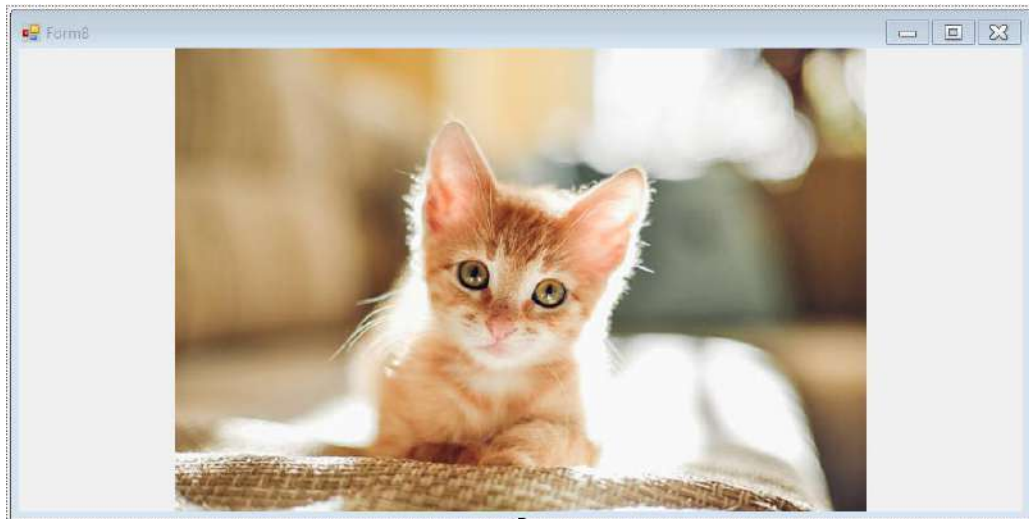
# Background Images for Forms

- **Stretch:** The image is resized both horizontally and vertically to fit in the form. If the image is resized more in one direction than the other, it appears stretched.
- **Zoom:** The image is uniformly resized to fit in the form without losing its original aspect ratio. This causes the image to be resized without appearing stretched.

Stretch



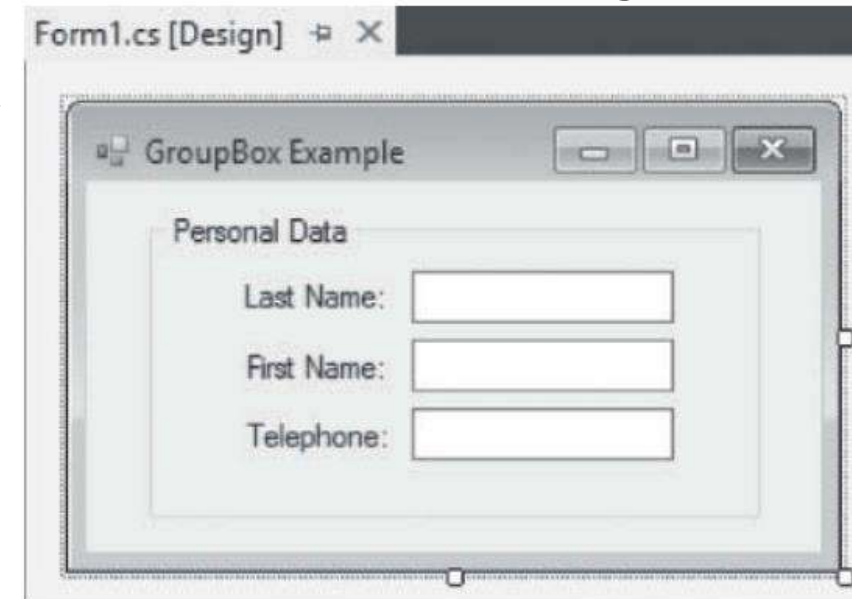
Zoom



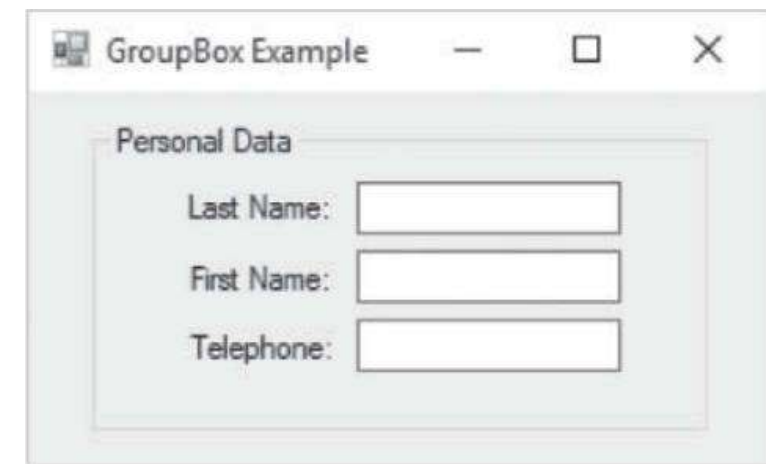
# GroupBox Control

- A GroupBox control is a rectangular control that appears with a thin border and an optional title in its upper-left corner.
- It is a container that can hold other controls and found in the Containers section of the Toolbox.
- When you create a GroupBox control, you can set its Text property to the title that you want displayed in the GroupBox's upper-left corner.
- If you don't want a title displayed on the GroupBox, you can clear the contents of its Text property.
- The Text property of the GroupBox control shown on the right is set to Personal Data and several other controls are inside the GroupBox.

The form in the Designer

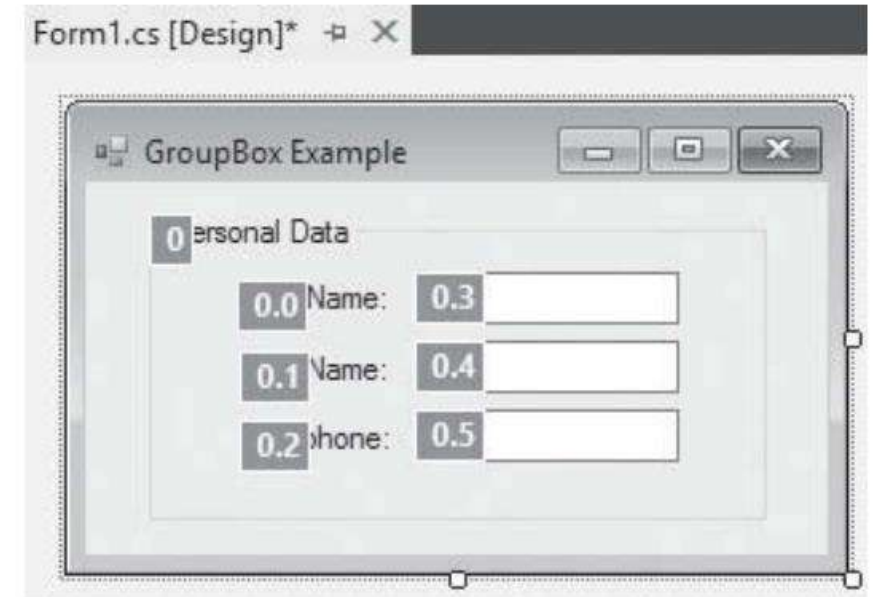


The form at run time



# GroupBox Tab Order

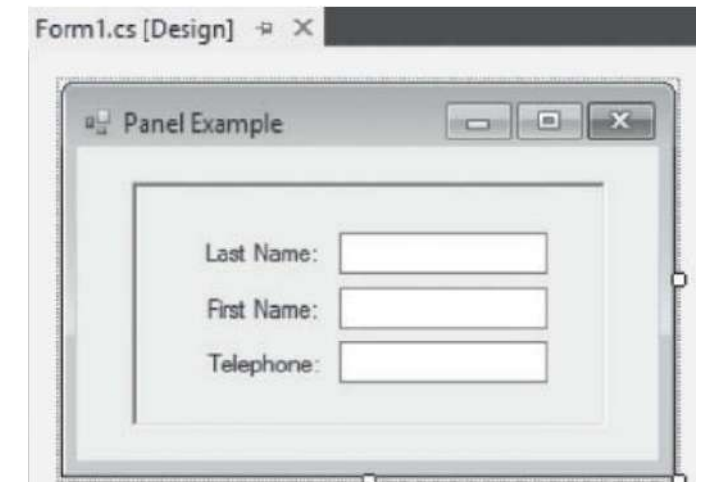
- The value of a control's TabIndex property is handled differently when the control is placed inside a GroupBox control.
- GroupBox controls have their own TabIndex property, and the TabIndex value of the controls inside the group box are relative to the GroupBox control's TabIndex property.
- The TabIndex properties of the controls inside the group box will not appear this way in the Properties window. They will appear as 0, 1, 2, and so forth.
- This figure shows a GroupBox control displayed in tab order selection mode.



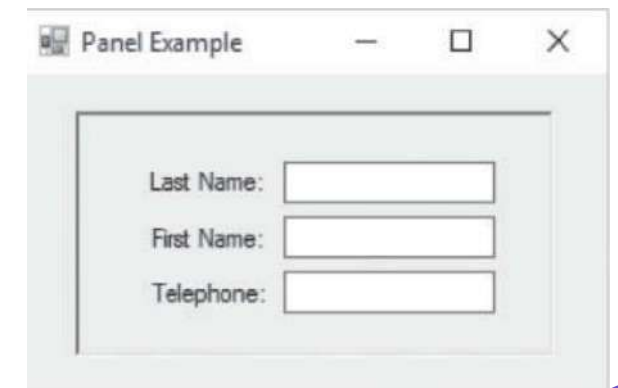
# Panel

- A Panel control is a rectangular container for other controls, like a GroupBox.
- There are several primary differences between a Panel and GroupBox:
  - A Panel cannot display a title and does not have a Text property.
  - A Panel's border can be specified by its BorderStyle property. The available settings are None, FixedSingle, and Fixed3D. The property is set to None by default.
- An example of a form with a Panel is shown on the right. The Panel's BorderStyle property is set to Fixed3D.

The form in the Designer



The form at run time



THANK YOU